

Tugas Deliverable 2

II3240 Rekayasa Sistem dan Teknologi Informasi

GATEKEEPER-AI

**Sistem Kontrol Akses Ruangan & Presensi Terpusat
Berbasis Real-Time Scheduling**



Disusun oleh

Kelompok 5

Zheannetta Apple Haihando	18223105
Muhammad Aymar Barkhaya	18223051
Nawaf Amjad Rizqi Aldaha Ismai	18223073
Aliya Harta Ary Utama	18223081
I Nyoman Rai Dharma Wiguna	18223083

**PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG**

2026

DAFTAR ISI

DAFTAR ISI.....	1
DAFTAR GAMBAR.....	3
DAFTAR TABEL.....	4
1. ADVANCED DEVELOPMENT.....	1
1.1 Requirement Analysis.....	1
1.1.1 System Functional Specification.....	1
1.1.2 Requirement Derivation.....	3
1.1.2.1 Requirement Derivation for System.....	4
1.1.2.2 Requirement Derivation for Website and Web Service.....	4
1.1.2.3 Requirement Derivation for Mobile App.....	5
1.1.2.4 Requirement Derivation for IoT.....	6
1.1.3 Hubungan dengan Persyaratan Operasional.....	6
1.1.4 Hubungan dengan Predecessor System.....	9
1.1.5 Identifikasi Komponen yang Memerlukan Pengembangan.....	10
1.1.6 Analisis Risiko.....	12
1.1.7 Identifikasi Rencana Mitigasi dan Estimasi Biaya / Upaya yang Diperlukan.....	13
1.1.8 Pengembangan Komponen Baru.....	14
1.2 Functional Analysis and Design.....	15
1.2.1 Critical Characteristic of System Functional Elements.....	15
1.2.2 Komponen yang Sangat Kompleks.....	16
1.2.3 Lingkungan Sistem yang Tidak Terdefinisi dengan Baik.....	17
1.3 Functional Design.....	18
1.3.1 System Interaction.....	19
1.4 Prototype Development.....	21
1.4.1 Mobile-App Design.....	21
1.4.2 Mobile-App Use Case.....	26
1.4.3 Website Design.....	27
1.4.4 Website Use Case.....	33
1.4.5 IoT Design.....	35
1.4.6 IoT Use Case.....	37
1.4.7 IoT Circuit Design.....	39
1.5 Development Testing.....	40
1.5.1 Test Planning.....	40
1.5.1.1 Tujuan Pengujian (Test Objectives).....	40
1.5.1.2 Overview Test Plans.....	40
1.5.2 Testing Environment.....	41
1.5.2.1 Perangkat Keras (Hardware).....	41
1.5.2.2 Perangkat Lunak (Software).....	42
1.5.2.3 Kondisi Lingkungan Fisik.....	42
1.5.3 Test Identification & Criteria.....	42
1.5.4 Testing Methodology.....	43

1.5.4.1 Unit Testing (Pengujian Terisolasi).....	43
1.5.4.2 Black-box Testing (Pengujian Fungsional).....	44
1.5.4.3 Stress & Load Testing (Pengujian Beban).....	44
1.5.4.4 Security & Vulnerability Testing (Pengujian Keamanan).....	44
1.5.5 Test Scenario.....	44
1.5.5.1 Langkah-Langkah Pengujian.....	45
1.5.5.2 Overview Test Scenario.....	45
1.5.6 Performance Metrics.....	46
1.5.7 Test Result Evaluation.....	47
1.5.7.1 Klasifikasi Status Kelulusan.....	47
1.5.7.2 Analisis Akar Masalah (Root Cause Analysis).....	47
1.5.7.3 Penentuan Iterasi.....	48
1.5.8 Failure and Recovery Procedure.....	48
1.5.8.1 Skenario Kegagalan dan Tindakan Pemulihan.....	48
1.5.8.2 Mekanisme Sinkronisasi Pasca Pemulihan.....	49
1.5.9 Development Risk Mitigation.....	49
2. PENUTUP.....	51
LAMPIRAN.....	52
REFERENSI.....	53

DAFTAR GAMBAR

Gambar 1.3.1 Diagram Interaksi Sistem GATEKEEPER-AI.....	19
Gambar 1.4.1.1 Dashboard Mahasiswa.....	22
Gambar 1.4.1.2 Rincian Mata Kuliah Mahasiswa.....	22
Gambar 1.4.1.3 Detail Presensi Mata Kuliah Mahasiswa.....	23
Gambar 1.4.1.4 Dashboard Dosen.....	23
Gambar 1.4.1.5 Rincian Mahasiswa dalam Kelas.....	24
Gambar 1.4.1.6 Rincian Mata Kuliah yang Diampu Dosen.....	24
Gambar 1.4.1.7 Rincian History Presensi Mata Kuliah yang Diampu Dosen.....	25
Gambar 1.4.1.8 Profile User.....	25
Gambar 1.4.1.9 Notifikasi User.....	26
Gambar 1.4.2 Diagram Mobile-App Use Case.....	27
Gambar 1.4.3.1 Splash Screen.....	29
Gambar 1.4.3.2 Login.....	29
Gambar 1.4.3.3 Dashboard Mahasiswa.....	30
Gambar 1.4.3.4 Daftar Jadwal Mata Kuliah Mahasiswa.....	30
Gambar 1.4.3.5 Rincian Kehadiran Kelas Mahasiswa.....	30
Gambar 1.4.3.6 Dashboard Dosen.....	31
Gambar 1.4.3.7 Jadwal Kelas Dosen.....	31
Gambar 1.4.3.8 Rincian Kehadiran Kelas Dosen.....	32
Gambar 1.4.3.9 Daftar Mahasiswa.....	32
Gambar 1.4.3.10 Profil Pengguna.....	32
Gambar 1.4.3.11 Notifikasi.....	33
Gambar 1.4.4 Diagram Website Use Case.....	35
Gambar 1.4.5.1 IoT Design Door Lock.....	36
Gambar 1.4.5.2 IoT Design Door Swing.....	37
Gambar 1.4.6. Use Case Diagram IoT Gatekeeper-AI.....	39
Gambar 1.4.7. Rancangan IoT Circuit Design.....	39

DAFTAR TABEL

Tabel 1.1.1 Spesifikasi Fungsional Sistem Gatekeeper-AI.....	1
Tabel 1.1.2.1 Requirement Derivation for System Gatekeeper-AI.....	4
Tabel 1.1.2.2 Requirement Derivation for Website and Web Service System Gatekeeper-AI.....	5
Tabel 1.1.2.3 Requirement Derivation for Mobile App Gatekeeper-AI.....	5
Tabel 1.1.2.4 Requirement Derivation for IoT.....	6
Tabel 1.1.3 Matriks Traceability Requirement terhadap Persyaratan Operasional.....	7
Tabel 1.1.4 Hubungan dengan Sistem Pendahulu.....	9
Tabel 1.1.5 Identifikasi Komponen yang Memerlukan Pengembangan.....	10
Tabel 1.1.6 Analisis Risiko.....	12
Tabel 1.1.7 Identifikasi Rencana Mitigasi dan Estimasi Biaya / Upaya yang Diperlukan.....	14
Tabel 1.1.8 Pengembangan Komponen Baru.....	14
Tabel 1.2.1 Karakteristik Kritis pada Fungsi Elemen Sistem GATEKEEPER-AI.....	15
Tabel 1.4.2 Spesifikasi Mobile Use Case pada GATEKEEPER-AI.....	26
Tabel 1.4.4 Spesifikasi Website Use Case pada GATEKEEPER-AI.....	33
Tabel 1.4.6.1 Aktor dan Sistem yang Terlibat pada IoT GATEKEEPER-AI.....	38
Tabel 1.4.6.2 Spesifikasi IoT Use Case pada GATEKEEPER-AI.....	38
Tabel 1.5.1 Tabel Testing Plan pada GATEKEEPER-AI.....	41
Tabel 1.5.3 Tabel Test Identification & Criteria pada GATEKEEPER-AI.....	43
Tabel 1.5.5.1 Tabel Skenario & Langkah-langkah Pengujian pada GATEKEEPER-AI.....	45
Tabel 1.5.5 Tabel Skenario & Langkah-langkah Pengujian pada GATEKEEPER-AI.....	46
Tabel 1.5.8.1 Skenario Kegagalan dan Tindakan Pemulihan pada GATEKEEPER-AI.....	48
Tabel 1.5.9 Tabel Manajemen Risiko Development pada GATEKEEPER-AI.....	50

1. *ADVANCED DEVELOPMENT*

Fase Pengembangan Lanjutan berfokus pada pematangan konsep sistem terpilih dan teknologi kritis hingga titik di mana desain rekayasa skala penuh dapat dilanjutkan dengan risiko yang dapat diterima. Ini melibatkan analisis persyaratan terperinci, desain fungsional, pengembangan prototipe, dan pengujian pengembangan awal, semuanya didukung oleh strategi pengurangan risiko yang kuat.

1.1 *Requirement Analysis*

1.1.1 *System Functional Specification*

System Functional Specification mendefinisikan seluruh fungsi yang harus dipenuhi oleh sistem secara menyeluruh, diturunkan dari kebutuhan operasional dan fungsional yang telah diidentifikasi pada tahap Concept Development di Deliverable sebelumnya.

Tabel 1.1.1 Spesifikasi Fungsional Sistem Gatekeeper-AI

Kode	Kebutuhan Sistem	Deskripsi
F01	Sistem dapat mendeteksi dan mengekstraksi fitur wajah mahasiswa secara otomatis.	Sistem menggunakan kamera yang terpasang di pintu masuk ruang kelas untuk menangkap citra wajah mahasiswa secara real-time. Edge engine pada laptop melakukan preprocessing (image enhancement, liveness detection) dan ekstraksi face embedding sebelum dikirim ke server pusat melalui REST API dalam format JSON.
F02	Sistem dapat mengenali identitas mahasiswa berdasarkan data biometrik wajah.	Server pusat menjalankan model face recognition untuk mencocokkan face embedding yang diterima dengan database mahasiswa (SIX). Sistem mengembalikan NIM mahasiswa apabila pencocokan berhasil, atau status "Tidak Dikenali"

		apabila gagal. Akurasi pengenalan minimal 98% dengan $FAR \leq 1\%$ dan $FRR \leq 5\%$.
F03	Sistem dapat memvalidasi hak akses mahasiswa berdasarkan jadwal perkuliahan granular.	Core Engine pada server pusat memverifikasi apakah NIM yang teridentifikasi memiliki jadwal perkuliahan yang sesuai dengan ruangan dan waktu saat ini. Validasi mencakup kombinasi spesifik hari, jam, dan kode ruangan yang disinkronisasi dari SIX setiap 24 jam.
F04	Sistem dapat mengontrol akses pintu ruang kelas secara otomatis.	Berdasarkan hasil validasi, server pusat mengirimkan instruksi OPEN atau LOCK ke mikrokontroler ESP32 melalui REST API. ESP32 kemudian menggerakkan solenoid door lock untuk membuka atau mengunci pintu. Magnetic reed sensor memantau status fisik pintu secara berkelanjutan.
F05	Sistem dapat mencatat data presensi mahasiswa secara otomatis.	Setiap kali mahasiswa berhasil memasuki ruang kelas, sistem secara otomatis mencatat NIM, timestamp kedatangan, dan kode ruangan ke dalam Granular Database di server pusat. Data presensi dijamin integritasnya 100% dengan mekanisme local caching pada edge device saat koneksi terputus.
F06	Sistem dapat memantau aktivitas keluar-masuk mahasiswa selama perkuliahan berlangsung.	Sistem merekam setiap kejadian keluar dan masuk mahasiswa sepanjang durasi sesi perkuliahan menggunakan data log dari magnetic reed sensor dan akses solenoid. Setiap event dicatat dengan timestamp akurat untuk keperluan kalkulasi durasi kehadiran.

F07	Sistem dapat menentukan status presensi mahasiswa secara otomatis berdasarkan durasi kehadiran.	Modul Attendance Monitoring pada server pusat menerapkan logika time-threshold 30 menit. Apabila mahasiswa meninggalkan ruang kelas dan tidak kembali dalam 30 menit, status presensinya secara otomatis diubah dari "Hadir" menjadi "Alpha". Timer di-reset apabila mahasiswa kembali masuk sebelum threshold terpenuhi.
F08	Sistem dapat menyediakan dashboard monitoring kehadiran secara real-time kepada dosen.	Antarmuka pengguna (mobile app dan web dashboard) menampilkan status kehadiran seluruh mahasiswa pada sesi perkuliahan yang sedang berlangsung. Data diperbarui secara otomatis dengan refresh rate ≤ 5 detik melalui koneksi ke server pusat via REST API.
F09	Sistem dapat memungkinkan dosen melakukan remote override kontrol pintu.	Dosen dapat mengirimkan perintah buka atau kunci pintu secara manual melalui fitur remote override pada aplikasi mobile. Perintah diteruskan dari aplikasi ke server pusat, kemudian ke ESP32, dan dieksekusi dalam waktu ≤ 5 detik secara end-to-end.
F10	Sistem dapat menyimpan dan menyinkronisasi seluruh data presensi ke database terpusat.	Seluruh log akses, data presensi, dan perubahan status kehadiran disimpan secara terpusat di Granular Database pada server pusat. Sinkronisasi data dari edge device ke server dilakukan dalam waktu ≤ 2 detik. Data tidak hilang saat terjadi gangguan koneksi berkat mekanisme local caching.

1.1.2 Requirement Derivation

Requirement derivation dilakukan untuk meninjau kembali skenario-skenario yang akan dihadapi oleh sistem yang mencakup persyaratan terkait kompatibilitas,

reliabilitas, dan faktor lingkungan. Berikut merupakan requirement derivation rancangan sistem Gatekeeper-AI

1.1.2.1 *Requirement Derivation for System*

Untuk setiap tahapan dari siklus hidup sistem yang telah dirancang sebelumnya, diperlukan penurunan kebutuhan sistem yang relevan. Berikut merupakan tabel penurunan *requirement derivation for system*.

Tabel 1.1.2.1 *Requirement Derivation for System Gatekeeper-AI*

Kode	Requirement
R-1	Sistem dapat mengidentifikasi mahasiswa berdasarkan wajah dengan akurasi > 95%.
R-2	Sistem dapat melakukan validasi hak akses mahasiswa berdasarkan kombinasi NIM, hari, jam dan ruangan secara granular.
R-3	Sistem dapat mencatat setiap kejadian masuk dan keluar dengan timestamp ke database.
R-4	Sistem dapat mengubah status presensi mahasiswa dari hadir menjadi alpha secara otomatis apabila mahasiswa tidak kembali dalam 30 menit setelah keluar ruangan.
R-5	Sistem harus memiliki uptime >99% selama jam operasional kampus.
R-6	Sistem dapat beroperasi dalam mode offline dan melakukan sinkronisasi ulang ketika koneksi pulih tanpa kehilangan data.
R-7	Sistem dapat menyimpan data biometrik (wajah) secara lokal, tidak dikirim ke cloud dalam bentuk raw image.
R-8	Waktu total dari wajah terdeteksi hingga pintu terbuka tidak boleh melebihi 2 detik.
R-9	Sistem harus menolak semua percobaan akses menggunakan foto/video.
R-10	Sistem dapat menyinkronisasi data jawal dari SIX setiap 24 jam secara otomatis.
R-11	Sistem dapat menyimpan seluruh log akses.
R-12	Sistem dapat mendukung remote override oleh dosen untuk membuka/mengunci pintu secara manual.

1.1.2.2 *Requirement Derivation for Website and Web Service*

Berikut merupakan tabel penurunan requirement derivation for website and webservice dari sistem Gatekeeper-AI

Tabel 1.1.2.2 *Requirement Derivation for Website and Web Service System Gatekeeper-AI*

Kode	Requirement
R-1	<i>Website dan web service system</i> harus menyediakan endpoint untuk menerima data NIM dan mengembalikan respon izin/tolak akses dalam waktu < 1 detik.
R-2	<i>Website dan web service system</i> dapat menyimpan <i>granular database</i> jadwal yang mencakup: NIM mahasiswa, kode mata kuliah, hari, jam mulai, jam selesai, dan kode ruangan.
R-3	<i>Website dan web service system</i> harus menyediakan endpoint untuk sinkronisasi jadwal dari SIX.
R-4	<i>Website dan web service system</i> menampilkan informasi mengenai log akses lengkap (NIM, timestamp, status, ruangan) yang dapat difilter per sesi/tanggal ke pengguna.
R-5	<i>Website dan web service system</i> dapat menampilkan informasi mengenai persentase kehadiran dan status kehadiran pada jadwal kelas tersebut.
R-6	<i>Website dan web service system</i> dapat mengekspor data presensi yang tersimpan serta dapat meng- <i>override</i> untuk membuka/mengunci pintu.
R-7	<i>Website dan web service system</i> memiliki mekanisme error handling dengan auto-restart apabila terjadi error.

1.1.2.3 *Requirement Derivation for Mobile App*

Berikut merupakan tabel penurunan requirement derivation for mobile app dari sistem Gatekeeper-AI

Tabel 1.1.2.3 *Requirement Derivation for Mobile App Gatekeeper-AI*

Kode	Requirement
R-7	<i>Mobile app</i> harus compatible dengan android 8.0 dan IOS 13.0
R-8	<i>Mobile app</i> dapat mengirimkan notifikasi kepada dosen, menyediakan riwayat presensi dan menyediakan fungsi <i>override</i> .
R-9	<i>Mobile app</i> dapat memiliki autentikasi login dengan menggunakan kredensial yang terverifikasi.

Kode	Requirement
R-10	<i>Mobile app</i> harus memiliki UI/UX yang intuitif sehingga > 9-% pengguna dapat mengoperasikan fitur utama tanpa pelatihan khusus.

1.1.2.4 Requirement Derivation for IoT

Berikut merupakan tabel penurunan requirement derivation for IoT dari sistem Gatekeeper-AI

Tabel 1.1.2.4 Requirement Derivation for IoT

Kode	Requirement
R-11	<i>IoT</i> pada modul kamera mampu menangkap citra wajah dan menjalankan proses deteksi saat mahasiswa berada dalam jarak 0,2 - 1 meter.
R-12	<i>IoT</i> pada mikrokontroler ESP32 dapat mengaktifkan solenoid door lock setelah menerima instruksi dari server.
R-13	Perangkat <i>IoT</i> mampu menyimpan log presensi secara lokal apabila koneksi Wifi terputus.
R-14	Perangkat IoT mampu beroperasi secara kontinu dengan minim error.

1.1.3 Hubungan dengan Persyaratan Operasional

Berikut merupakan pemetaan hubungan ketertelusuran (traceability) antara requirement yang telah diturunkan pada 1.1.2 dengan persyaratan operasional (Operational Objectives) yang ditetapkan pada tahap Concept Development di deliverable 1. Pemetaan ini memastikan bahwa setiap requirement yang dikembangkan memiliki justifikasi langsung terhadap kebutuhan operasional sistem GATEKEEPER-AI di lingkungan perkuliahan kampus.

Persyaratan operasional sistem GATEKEEPER-AI dirangkum dalam 5 objektif utama (O01–O05) sebagaimana ditetapkan dalam Tabel 2.1.3 di deliverable 1:

Kode	Requirement
O01	Mengembangkan sistem presensi yang mampu memastikan kehadiran mahasiswa secara lebih akurat.

Kode	Requirement
O02	Menerapkan mekanisme verifikasi identitas mahasiswa untuk meminimalkan manipulasi presensi.
O03	Menyediakan sistem yang dapat membantu dosen memverifikasi kehadiran mahasiswa secara otomatis.
O04	Mengembangkan sistem presensi yang terintegrasi dengan mekanisme akses ruang kelas.
O05	Merancang sistem yang efisien dan tidak menghambat proses masuk kelas.

Berikut adalah matriks ketertelusuran antara requirement hasil derivasi (1.1.2) dengan persyaratan operasional tersebut:

Tabel 1.1.3 Matriks Traceability Requirement terhadap Persyaratan Operasional

Kode	Requirement	O01	O02	O03	O04	O05
System						
R-1	Sistem dapat mengidentifikasi mahasiswa berdasarkan wajah dengan akurasi > 95%	✓	✓			
R-2	Sistem dapat melakukan validasi hak akses berdasarkan NIM, hari, jam, dan ruangan secara granular	✓	✓		✓	
R-3	Sistem dapat mencatat setiap kejadian masuk dan keluar dengan timestamp ke database	✓		✓		
R-4	Sistem dapat mengubah status presensi dari hadir menjadi alpha secara otomatis apabila mahasiswa tidak kembali dalam 30 menit	✓		✓		
R-5	Sistem harus memiliki uptime >99% selama jam operasional kampus	✓			✓	✓
R-6	Sistem dapat beroperasi dalam mode offline dan sinkronisasi ulang saat koneksi pulih tanpa kehilangan data	✓			✓	✓
R-7	Sistem dapat menyimpan data biometrik secara lokal, tidak dikirim ke cloud dalam bentuk raw image		✓			

R-8	Waktu total dari wajah terdeteksi hingga pintu terbuka tidak melebihi 2 detik				✓	✓
R-9	Sistem harus menolak semua percobaan akses menggunakan foto/video		✓			
R-10	Sistem dapat menyinkronisasi data jadwal dari SIX setiap 24 jam secara otomatis	✓			✓	
R-11	Sistem dapat menyimpan seluruh log akses	✓		✓		
R-12	Sistem dapat mendukung remote override oleh dosen			✓	✓	
Website & Web Service						
R-1	Endpoint menerima data NIM dan mengembalikan respons izin/tolak dalam waktu < 1 detik				✓	✓
R-2	Menyimpan granular database jadwal (NIM, mata kuliah, hari, jam, ruangan)	✓			✓	
R-3	Endpoint untuk sinkronisasi jadwal dari SIX	✓			✓	
R-4	Menampilkan log akses lengkap yang dapat difilter per sesi/tanggal	✓		✓		
R-5	Menampilkan persentase dan status kehadiran per jadwal kelas	✓		✓		
R-6	Ekspor data presensi dan override buka/kunci pintu			✓	✓	
R-7	Error handling dengan auto-restart apabila terjadi error				✓	✓
Mobile App						
R-7	Compatible dengan Android 8.0 dan iOS 13.0			✓		✓
R-8	Notifikasi kepada dosen, riwayat presensi, dan fungsi override			✓	✓	
R-9	Autentikasi login dengan kredensial terverifikasi		✓	✓		
R-10	UI/UX intuitif sehingga >90% pengguna dapat mengoperasikan fitur utama tanpa pelatihan			✓		✓

IoT						
R-11	Kamera mampu menangkap citra wajah pada jarak 0,2–1 meter	✓	✓			✓
R-12	ESP32 dapat mengaktifkan solenoid door lock setelah menerima instruksi dari server				✓	✓
R-13	Perangkat IoT mampu menyimpan log presensi secara lokal saat Wi-Fi terputus	✓			✓	
R-14	Perangkat IoT mampu beroperasi secara kontinu dengan minim error	✓			✓	✓

Berdasarkan matriks di atas, seluruh requirement yang diturunkan pada 1.1.2 dapat ditelusuri kembali ke minimal satu persyaratan operasional. Objektif O01 (akurasi kehadiran) dan O04 (integrasi akses ruang kelas) memiliki cakupan terluas karena menjadi inti dari arsitektur sistem. Sementara O02 (pencegahan manipulasi presensi) secara khusus dipenuhi oleh requirement terkait face recognition, liveness detection, dan keamanan data biometrik.

1.1.4 Hubungan dengan *Predecessor System*

Sebelum GATEKEEPER-AI dikembangkan, proses pencatatan kehadiran mahasiswa di ITB dilakukan melalui tiga mekanisme predecessor: daftar hadir manual, tandai hadir SIX, dan scan QR Code Edunex. Ketiga sistem tersebut memiliki keterbatasan fundamental yang menjadi landasan pengembangan GATEKEEPER-AI.

Tabel 1.1.4 Hubungan dengan Sistem Pendahulu

Aspek	Sistem Pendahulu	Sistem Baru (GATEKEEPER-AI)
Verifikasi Identitas	Dilakukan secara manual atau berbasis kepemilikan perangkat/tanda tangan tanpa verifikasi fisik	Otomatis menggunakan face recognition berbasis AI dengan akurasi $\geq 95\%$
Pencegahan Titip Absen	Tidak tersedia. Mahasiswa dapat menitipkan presensi kepada teman	Tidak memungkinkan karena identifikasi berbasis biometrik wajah yang unik per individu

Pemantauan Durasi Kehadiran	Tidak tersedia. Presensi hanya dicatat satu kali	Berkelanjutan sepanjang sesi dengan logika time-threshold 30 menit untuk deteksi ghosting kuliah
Monitoring Real-Time oleh Dosen	Tidak tersedia. Data kehadiran lengkap umumnya baru tersedia di akhir sesi	Tersedia melalui dashboard dan mobile app dengan refresh rate ≤ 5 detik
Sinkronisasi Jadwal Akademik	Tidak terintegrasi dengan data jadwal SIX secara otomatis	Sinkronisasi otomatis dari SIX setiap 24 jam untuk validasi hak akses granular
Keamanan Data Presensi	Rentan manipulasi (foto QR Code, tanda tangan palsu)	Log akses 100% tercatat dengan timestamp, data biometrik disimpan lokal

1.1.5 Identifikasi Komponen yang Memerlukan Pengembangan

Sebelum tahap perancangan sistem, pengembang perlu melakukan identifikasi terkait komponen mana saja yang sudah terbukti dan siap digunakan, yang masih perlu diuji, dan yang harus dikembangkan lebih lanjut. Berikut merupakan tabel penilaian kematangan komponen yang digunakan pengembang sebagai acuan utama.

Tabel 1.1.5 Identifikasi Komponen yang Memerlukan Pengembangan

Komponen	Berhasil / Sebagian / Belum Berhasil Diproduksi	Evaluasi Fungsional	Evaluasi Fisik	Antarmuka dan Interaksi	Butuh Uji Eksperimen / Tidak Butuh
ESP32 Mikrokontroller	Berhasil	Terbukti menjalankan fungsi kontrol aktuator dan komunikasi Wi-Fi via REST API	Stabil dan efisien daya, banyak digunakan dalam ekosistem IoT	Terintegrasi dengan relay, solenoid, dan magnetic reed sensor	X

Kamera OV2640	Sebagian	Mampu menangkap citra wajah real-time, namun akurasi bergantung pada kondisi pencahayaan	Perlu diuji pada jarak operasional 0,2–1 meter di ruang kelas	Kualitas frame yang dikirim ke edge engine perlu divalidasi	✓
Solenoid Door Lock	Berhasil	Terbukti membuka/mengunci pintu berdasarkan sinyal relay dari ESP32	Perlu validasi ketahanan penggunaan intensif sepanjang hari	Waktu respon aktual perlu diuji (target 0,5–4,0 detik)	✓
Model Face Recognition (AI)	Belum Berhasil	Akurasi $\geq 98\%$ dan FAR $\leq 1\%$ belum dibuktikan pada kondisi nyata dataset mahasiswa ITB	Performa bergantung pada kualitas hardware dan pencahayaan ruangan	Pipeline deteksi, ekstraksi embedding, dan pencocokan database perlu divalidasi end-to-end	✓
Liveness Detection (Anti-Spoofing)	Belum Berhasil	Belum diimplementasikan diperlukan untuk mencegah akses via foto/video	Harus berjalan real-time tanpa menambah latensi signifikan	Wajib terintegrasi dalam pipeline face recognition sebelum hasil dikirim ke server	✓
Backend Server & Core Engine	Belum Berhasil	Logika validasi granular dan time-threshold 30 menit belum diimplementasikan	Harus mampu menangani hingga 100 permintaan verifikasi serentak	REST API antara server, edge device, dan ESP32 perlu dirancang dan diuji	✓

Granular Database (SIX Sync)	Belum Berhasil	Skema jadwal granular dan sinkronisasi otomatis dari SIX belum dibangun	Bergantung pada ketersediaan API ekspor data jadwal dari SIX ITB	Respons validasi harus < 1 detik	✓
Mobile App (Dashboard Dosen)	Belum Berhasil	Fitur monitoring, notifikasi, riwayat presensi, dan remote override belum dikembangkan	Harus kompatibel dengan Android 8.0 dan iOS 13.0	Latensi refresh dashboard (≤ 5 detik) dan override (≤ 5 detik) perlu diuji	✓

1.1.6 Analisis Risiko

Berdasarkan hasil identifikasi komponen pengembangan pada tahap sebelumnya, diperlukan analisis risiko lebih lanjut agar mendapat gambaran mengenai dampak potensial yang dapat terjadi dan biaya yang kira-kira dibutuhkan untuk menanganinya. Berikut merupakan tabel analisis risiko untuk setiap komponen yang masih perlu dilakukan pengujian lebih lanjut.

Tabel 1.1.6 Analisis Risiko

Komponen	Risiko Teknis	Dampak Potensial	Pengujian Tambahan
Kamera OV2640	Akurasi tangkapan wajah bergantung pada kondisi pencahayaan, kamera harus dapat mendeteksi dengan jarak 0,2 - 1 meter.	Wajah tidak terdeteksi/terbaca dengan baik sehingga proses identifikasi terganggu.	Melakukan uji kejernihan citra pada jarak 0,2–1 meter, dan uji pada variasi pencahayaan ruang kelas.
Solenoid Door Lock	Ketahanan untuk penggunaan intensif sepanjang hari belum tervalidasi, waktu respon aktual masih perlu diuji.	Pintu terlambat terbuka/terkunci atau gagal mengeksekusi perintah akses	Melakukan uji latensi aktuator dan uji ketahanan operasi berulang/intensif.

Model Face Recognition (AI)	Akurasi $\geq 98\%$ dan FAR $\leq 1\%$ belum dibuktikan pada kondisi nyata, performa bergantung pada hardware dan pencahayaan, pipeline end-to-end belum tervalidasi.	Model salah mengidentifikasi, menolak/menerima akses secara keliru.	Melakukan uji akurasi identifikasi wajah, dan uji validasi <i>end to end</i> dari deteksi ekstraksi pencocokan wajah.
Liveness Detection (Anti-Spoofing)	Sistem belum mengimplementasikan anti-spoofing, sistem harus berjalan real-time tanpa menambah latensi signifikan, sistem wajib terintegrasi sebelum hasil dikirim ke server.	Sistem rentan terhadap akses menggunakan foto/video	Melakukan simulasi spoofing dengan foto/video; dan mengukur pengaruh modul terhadap latensi sistem.
Backend Server & Core Engine	Logika validasi granular dan time-threshold 30 menit belum diimplementasikan, REST API antar subsistem belum dirancang dan diuji.	Sistem memvalidasi hak akses gagal/lambat, perubahan status presensi tidak konsisten dan terjadi <i>bottleneck</i> saat <i>high traffic</i>	Melakukan uji integrasi REST API, dan uji logika threshold 30 menit
Granular Database (SIX Sync)	Skema jadwal granular dan sinkronisasi otomatis dari SIX belum ada, sistem bergantung pada ketersediaan API ekspor jadwal SIX, sistem harus dapat memvalidasi < 1 detik	Sistem memberikan hak akses yang tidak sesuai jadwal dan sinkronisasi data akademik terlambat atau tidak konsisten.	Melakukan uji sinkronisasi SIX setiap 24 jam, dan uji kecepatan respon validasi database.
Mobile App (Dashboard Dosen)	Monitoring, notifikasi, riwayat presensi, dan remote override belum dikembangkan.	Dosen tidak dapat memantau kelas atau melakukan override secara andal.	Melakukan uji kompatibilitas perangkat, uji refresh dashboard ≤ 5 detik dan uji override <i>end to end</i> ≤ 5 detik.

1.1.7 Identifikasi Rencana Mitigasi dan Estimasi Biaya / Upaya yang Diperlukan

Berdasarkan hasil analisis risiko yang telah dilakukan, diperlukan analisis lanjutan terhadap rencana mitigasi untuk masing-masing risiko yang berpotensi untuk

terjadi serta estimasi biaya atau upaya yang dapat dilakukan. Berikut merupakan tabel ringkasan dari risiko utama, alternatif solusi dan estimasi biaya/upaya yang dapat dilakukan.

Tabel 1.1.7 Identifikasi Rencana Mitigasi dan Estimasi Biaya / Upaya yang Diperlukan

Risiko Utama	Alternatif Solusi / Mitigasi	Estimasi Biaya / Upaya
Kamera tidak optimal saat pencahayaan ruang kelas berubah atau posisi wajah tidak ideal.	Menambah/ mengganti camera module yang lebih sesuai untuk AI capture, melakukan kalibrasi posisi kamera, dan menguji ulang pada jarak 0,2–1 meter serta variasi pencahayaan ruang kelas.	Rp.175.000/unit kamera
Solenoid door lock berisiko lambat atau kurang andal saat penggunaan intensif.	Mengganti atau menyesuaikan solenoid door lock, mengecek relay module, serta mengoptimalkan instruksi dari ESP32 ke aktuator.	Rp.95.000/unit
Kontrol aktuator terganggu karena mikrokontroler atau jalur kontrol bermasalah	Menyediakan ESP32 cadangan dan melakukan integrasi ulang komunikasi REST API dengan aktuator.	Rp.75.000/unit
Pemantauan status pintu kurang andal	Menambahkan atau mengganti magnetic door sensor untuk memonitor kondisi buka/tutup pintu secara real-time	Rp.25.000/unit

1.1.8 Pengembangan Komponen Baru

Berikut merupakan tabel pengembangan komponen baru berdasarkan hasil analisis risiko dan tingkat kematangan dari sisi fungsional, fisik dan metode produksinya.

Tabel 1.1.8 Pengembangan Komponen Baru

Komponen	Design Approach	Development	Validation
Model Face Recognition (AI)	Pemrosesan dilakukan melalui alur kamera, edge processing, ekstraksi embedding, server pusat untuk pencocokan dengan database mahasiswa.	Mengembangkan pipeline deteksi wajah, preprocessing, ekstraksi embedding, dan pencocokan identitas	Melakukan uji akurasi identifikasi wajah, dan uji validasi <i>end to end</i> dari deteksi ekstraksi pencocokan wajah.
Liveness Detection	Modul ditempatkan	Mengimplementasikan	Melakukan simulasi

(Anti-Spoofing)	pada tahap preprocessing di edge sebelum hasil dikirim ke server.	deteksi anti-spoofing agar mampu membedakan wajah asli dengan foto/video secara real-time	spoofing dengan foto/video; dan mengukur pengaruh modul terhadap latensi sistem.
Backend Server & Core Engine	Server pusat memvalidasi identitas, jadwal granular, status presensi, dan mengirim instruksi OPEN/LOCK ke ESP32 melalui REST API.	Mengembangkan logika validasi akses, logging, attendance monitoring, time-threshold 30 menit, dan API antar subsistem.	Melakukan uji integrasi REST API, dan uji logika threshold 30 menit.
Granular Database (SIX Sync)	Database terpusat menyimpan jadwal granular, log akses, data presensi, dan sinkronisasi data dari SIX setiap 24 jam	Membangun skema database jadwal dan mekanisme sinkronisasi otomatis dari SIX	Melakukan uji sinkronisasi SIX setiap 24 jam, dan uji kecepatan respon validasi database.
Mobile App	Pendekatan user centered design dengan dashboard monitoring real time, autentikasi aman, dan fitur remote override	Mengembangkan login, monitoring presensi, riwayat presensi, notifikasi, dan override pintu	Melakukan uji kompatibilitas perangkat, uji refresh dashboard ≤ 5 detik dan uji override <i>end to end</i> ≤ 5 detik.

1.2 Functional Analysis and Design

Sub-fase ini berfokus pada pendefinisian arsitektur fungsional sistem dan bagaimana elemen-elemennya berinteraksi agar dapat menjadi dasar untuk membuat desain terperinci.

1.2.1 Critical Characteristic of System Functional Elements

Karakteristik kritis digunakan untuk mengidentifikasi parameter utama yang menentukan keberhasilan setiap fungsi GATEKEEPER-AI. Karakteristik ini menjadi acuan dalam pengembangan, pengujian, dan validasi performa sistem.

Tabel 1.2.1 Karakteristik Kritis pada Fungsi Elemen Sistem GATEKEEPER-AI

No	System Functions	Functional Elements	Critical Characteristics
----	------------------	---------------------	--------------------------

1	Identifikasi wajah mahasiswa	Kamera + Edge Processing	Akurasi $\geq 95\%$, waktu deteksi ≤ 1 detik, robust terhadap pencahayaan
2	Ekstraksi dan pengiriman data wajah	Edge Device + REST API	Latensi pengiriman ≤ 1 detik, data dalam format JSON, keamanan data
3	Pengenalan identitas mahasiswa	Face Recognition Engine (Server)	Akurasi $\geq 98\%$, FAR $\leq 1\%$, FRR $\leq 5\%$
4	Validasi hak akses	Core Engine + Database SIX	Validasi jadwal real-time, respon ≤ 1 detik
5	Kontrol akses pintu	ESP32 + Solenoid Door Lock	Delay aktuator ≤ 1 detik, reliability $\geq 99\%$
6	Pencatatan presensi otomatis	Database + Logging System	Integritas data 100%, timestamp akurat
7	Monitoring aktivitas keluar-masuk	Sensor pintu (reed switch) + Backend	Deteksi event real-time, akurasi log $\geq 95\%$
8	Penentuan status kehadiran	Attendance Engine	Threshold waktu 30 menit, konsistensi logika
9	Dashboard monitoring	Web & Mobile App	Web & Mobile App
10	Keamanan sistem	Liveness Detection	Deteksi spoofing $\geq 95\%$, anti-fraud

1.2.2 Komponen yang Sangat Kompleks

Elemen	Komponen	Deskripsi
Perangkat Lunak	<i>Face Recognition Real-time System</i>	Koordinasi antara deteksi wajah, preprocessing (image enhancement & liveness detection), ekstraksi fitur, serta pengiriman data ke

		server harus dilakukan secara real-time dengan latensi rendah.
	Sistem Pemrosesan Terdistribusi	Proses dilakukan secara terdistribusi antara edge device dan server pusat sehingga developer tidak memiliki kontrol penuh terhadap latensi jaringan dan sinkronisasi data.
	Antara Pengguna (<i>Dashboard & Mobile App</i>)	Sistem harus menyajikan informasi presensi dan status akses secara real-time dalam bentuk yang intuitif dan mudah dipahami oleh dosen.
Sistem Dinamis	Interaksi Antar Subsystem (<i>Edge-Server-IoT</i>)	Terdapat kompleksitas dalam interaksi antara subsystem (kamera, edge processing, server, dan ESP32) yang harus sinkron dan konsisten dalam waktu nyata.
Perangkat Keras & IoT	ESP32 dan <i>Smart Lock Control</i>	Kontrol aktuator pintu harus memiliki respon cepat dan reabilitas tinggi, serta bergantung pada komunikasi jaringan dengan server.
Keamanan Sistem	<i>Liveness Detection & Anti-Spoofing</i>	Sistem harus mampu membedakan wajah asli dengan foto atau video secara real-time, yang akan menambahkan kompleksitas algoritma dan komputasi.

1.2.3 Lingkungan Sistem yang Tidak Terdefinisi dengan Baik

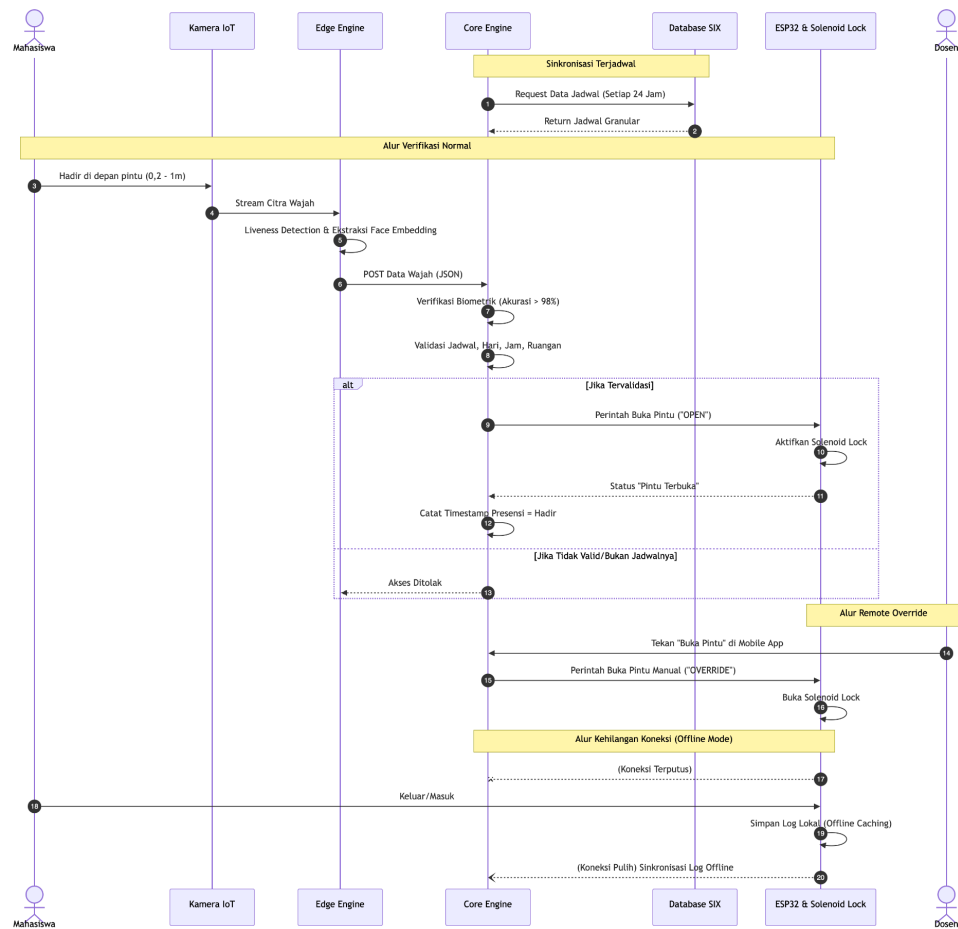
Kasus	Deskripsi	Tantangan

Variabilitas Fisik & Lingkungan	Sistem <i>face recognition</i> harus berfungsi dalam berbagai kondisi pencahayaan dan posisi pengguna di lingkungan ruang kelas	- Pencahayaan rendah/<i>backlight</i> - Posisi wajah tidak ideal - Pengguna mengenakan kacamata, masker, atau benda lainnya yang menghalangi wajah
Kondisi Jaringan	Sistem bergantung pada komunikasi antara <i>edge device</i> dan server untuk validasi dan sinkronisasi data	- Wi-Fi tidak stabil - Latensi jaringan tinggi - Potensi delay sinkronisasi data
Interaksi Pengguna-Alat	Mahasiswa berinteraksi dengan sistem secara cepat dan tanpa pelatihan khusus	- Perilaku pengguna tidak konsisten - Posisi wajah tidak sesuai kamera - Perlu UI/UX yang intuitif
Penggunaan Massal (<i>High-Traffic</i>)	Sistem digunakan oleh banyak mahasiswa dalam waktu bersamaan saat pergantian kelas	- <i>Bottleneck</i> pada pemrosesan - <i>Delay</i> respon sistem - Antrian identifikasi
Gangguan Keamanan (<i>Spoofing</i>)	Sistem berpotensi menghadapi upaya manipulasi menggunakan foto atau video wajah	- Deteksi <i>spoofing</i> kompleks - Risiko <i>false acceptance</i> - Kebutuhan <i>liveness detection</i>

1.3 Functional Design

Desain Fungsional (Functional Design) merupakan tahapan dalam rekayasa sistem yang bertujuan untuk menerjemahkan kebutuhan dan spesifikasi sistem menjadi sebuah arsitektur logis. Tahap ini berfokus pada pendefinisian apa yang harus dilakukan oleh sistem secara menyeluruh, serta memetakan aliran data (data flow), aliran kontrol (control flow), dan batasan operasional di antara berbagai elemen fungsional sebelum diimplementasikan ke dalam desain fisik.

1.3.1 System Interaction



Gambar 1.3.1 Diagram Interaksi Sistem GATEKEEPER-AI

Interaksi pada sistem GATEKEEPER-AI melibatkan berbagai komponen subsistem secara terintegrasi, mulai dari perangkat keras (IoT) di lapangan, pemrosesan lokal (*Edge Engine*), pemrosesan terpusat (*Core Engine*), hingga antarmuka pengguna (*Web/Mobile App*) dan sistem pihak ketiga (SIX ITB). Interaksi ini dibagi ke dalam empat alur skenario utama:

a. Alur Akses dan Verifikasi Kehadiran (Skenario Normal)

- i. **Pengambilan Citra:** Mahasiswa berdiri di depan pintu ruang kelas pada jarak 0,2 hingga 1 meter, dan sistem kamera (OV2640) menangkap citra wajah secara *real-time*.
- ii. **Pemrosesan Edge:** Aliran (*stream*) video dikirim ke *Edge Engine* lokal untuk dilakukan *preprocessing*, *liveness detection* (pencegahan *spoofing*), dan ekstraksi *face embedding*. Data hasil ekstraksi kemudian

dikirimkan ke *Core Engine* di Server Pusat melalui komunikasi REST API dalam format JSON.

- iii. **Validasi Server:** Server Pusat menjalankan model *Face Recognition* untuk mencocokkan wajah dengan *database* mahasiswa. Selanjutnya, *Core Engine* memvalidasi hak akses dengan membandingkan NIM yang terdeteksi terhadap jadwal perkuliahan spesifik (kombinasi hari, jam, dan ruangan) yang disinkronisasi dari sistem SIX ITB.
- iv. **Eksekusi Aktuator:** Jika akses diberikan, server mengirimkan instruksi "OPEN" ke mikrokontroler ESP32. ESP32 merespons dengan mengaktifkan *solenoid door lock* untuk membuka pintu, lalu sistem mencatat waktu kedatangan secara otomatis di *Granular Database*.

b. Alur Pemantauan dan Remote Override

- i. **Pemantauan *Real-Time*:** Aplikasi Web dan *Mobile App* milik dosen secara konstan meminta pembaruan data (dengan latensi ≤ 5 detik) melalui REST API ke Server Pusat untuk menampilkan daftar kehadiran dan status mahasiswa di kelas tersebut.
- ii. **Manual Override:** Dosen memiliki otoritas untuk melewati (*bypass*) sistem biometrik dengan memicu perintah buka/kunci pintu dari aplikasi *Mobile*. Instruksi *override* diteruskan dari aplikasi ke Server Pusat, lalu langsung menuju ESP32 untuk membuka pintu secara instan.

c. Alur Pemantauan Aktivitas Kelas

Selama sesi kuliah, *magnetic reed sensor* dan sistem *logging* mendeteksi interaksi fisik pada pintu. Sistem menerapkan logika toleransi (*time-threshold*). Apabila sistem mencatat seorang mahasiswa keluar ruangan dan tidak terdeteksi kembali masuk dalam kurun waktu 30 menit, status presensinya akan diubah otomatis dari "Hadir" menjadi "Alpha".

d. Alur Caching Luring

Jika ESP32 mendeteksi hilangnya koneksi Wi-Fi ke Server Pusat, unit IoT otomatis beralih ke mode luring (*offline*). Log aktivitas pintu dan presensi sementara disimpan secara lokal di dalam memori ESP32. Ketika koneksi stabil kembali, ESP32 akan menyinkronisasi ulang seluruh log secara utuh ke Server Pusat tanpa risiko kehilangan data.

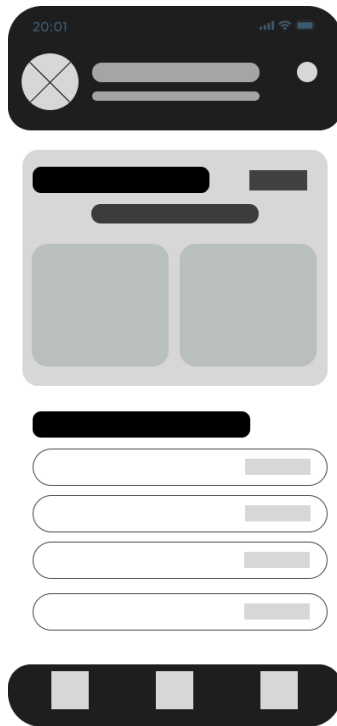
1.4 *Prototype Development*

Pengembangan Prototipe (Prototype Development) merupakan tahapan krusial dalam rekayasa sistem di mana spesifikasi kebutuhan dan desain fungsional diterjemahkan menjadi model awal yang wujudnya dapat dioperasikan dan diuji. Tujuan utama dari fase ini adalah untuk memvalidasi kelayakan teknis, mengevaluasi desain antarmuka (User Interface/User Experience), serta memverifikasi kelancaran integrasi antara perangkat lunak dan perangkat keras sebelum sistem diimplementasikan dalam skala penuh. Melalui pembuatan prototipe, pengembang dapat mengidentifikasi potensi kegagalan, latensi, atau ketidaksesuaian desain sedini mungkin sehingga risiko perbaikan berbiaya tinggi di masa depan dapat diminimalkan.

1.4.1 *Mobile-App Design*

Rancangan aplikasi mobile GATEKEEPER-AI dikembangkan dengan pendekatan *User-Centered Design* yang mengutamakan efisiensi dan kemudahan bagi dosen dengan mobilitas tinggi di lingkungan kampus. Dengan beberapa hal yang ditekankan seperti berikut:

- **Antarmuka Intuitif & Responsif:** UI didesain sesederhana mungkin agar lebih dari 90% pengguna dapat mengoperasikan fitur utama tanpa pelatihan khusus, serta kompatibel dengan sistem operasi Android 8.0 dan iOS 13.0 ke atas.
- **Dashboard Monitoring Real-Time:** Menampilkan status kehadiran mahasiswa pada sesi perkuliahan yang sedang berlangsung secara dinamis dengan tingkat pembaruan data (*refresh rate*) kurang dari 5 detik melalui koneksi REST API ke server pusat.
- **Kontrol Akses Terpusat:** Menyediakan tombol khusus untuk fitur *Remote Override* yang memungkinkan dosen membuka atau mengunci pintu ruang kelas secara instan dari jarak jauh.
- **Keamanan :** Aplikasi mendukung autentikasi login yang aman dengan menggunakan SSO Login ITB.



Gambar 1.4.1.1 Dashboard Mahasiswa



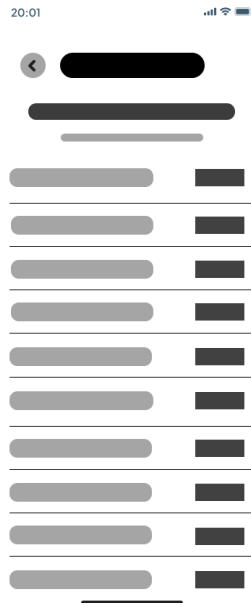
Gambar 1.4.1.2 Rincian Mata Kuliah Mahasiswa



Gambar 1.4.1.3 Detail Presensi Mata Kuliah Mahasiswa



Gambar 1.4.1.4 Dashboard Dosen



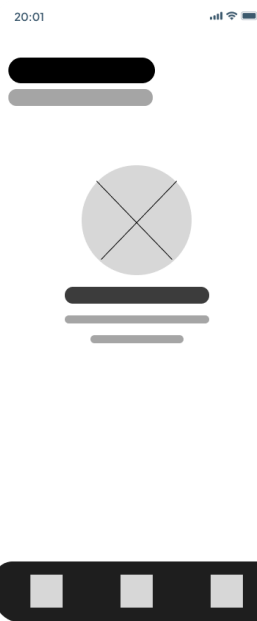
Gambar 1.4.1.5 Rincian Mahasiswa dalam Kelas



Gambar 1.4.1.6 Rincian Mata Kuliah yang Diampu Dosen



Gambar 1.4.1.7 Rincian History Presensi Mata Kuliah yang Diampu Dosen



Gambar 1.4.1.8 Profile User



Gambar 1.4.1.9 Notifikasi User

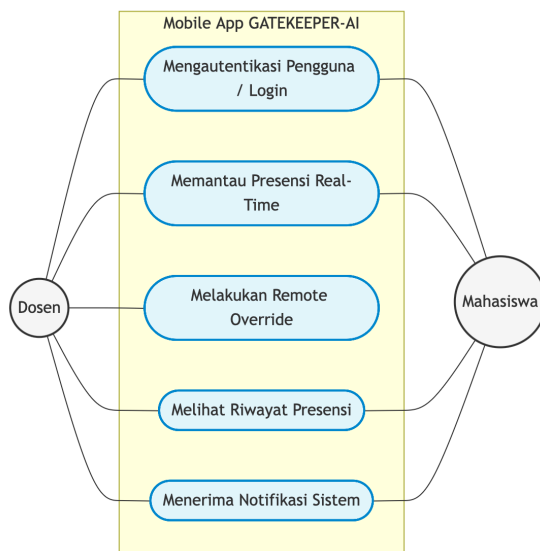
1.4.2 Mobile-App Use Case

Bagian ini mendefinisikan interaksi fungsional antara pengguna (Dosen dan Mahasiswa) sebagai aktor dengan aplikasi *mobile* dalam ekosistem GATEKEEPER-AI. Kedua aktor ini memiliki akses ke sebagian besar fitur yang sama, dengan perbedaan hak akses khusus pada kontrol fisik pintu ruangan. Dengan beberapa *use case* seperti berikut :

Tabel 1.4.2 Spesifikasi Mobile Use Case pada GATEKEEPER-AI

ID	Use Case Name	Req	Description
UC-Mobile -01	Mengautentikasi Pengguna (Login)	R-7, R-9	Dosen dan mahasiswa melakukan autentikasi menggunakan kredensial yang telah terverifikasi. Hal ini memastikan hanya pihak berwenang yang dapat mengakses kontrol pintu dan data akademik.
UC-Mobile -02	Memantau Presensi Real-Time	R-7, R-8, R-10	Dosen dan mahasiswa memantau pembaruan data kehadiran secara otomatis dengan UI/UX yang baik dan intuitif. Sistem menampilkan informasi <i>timestamp</i> kedatangan dan status kehadiran (Hadir/Alpha).

ID	Use Case Name	Req	Description
UC-Mobile-03	Melakukan Remote Override	R-7, R-8	Dosen menggunakan fungsi <i>override</i> untuk mengirimkan perintah buka atau kunci pintu secara manual kepada perangkat ESP32. Mahasiswa tidak memiliki akses ke fitur ini.
UC-Mobile-04	Melihat Riwayat Presensi	R-7, R-8	Dosen dan mahasiswa mengakses dan menyediakan log riwayat kehadiran dari sesi-sesi perkuliahan sebelumnya untuk keperluan evaluasi kelas atau pemantauan persentase akademik secara mandiri.
UC-Mobile-05	Menerima Notifikasi Sistem	R-7, R-8	Aplikasi mengirimkan notifikasi kepada pengguna. Mahasiswa menerima peringatan terkait batas waktu 30 menit sebelum status menjadi "Alpha", sementara dosen menerima pembaruan status kelas atau kendala perangkat IoT.



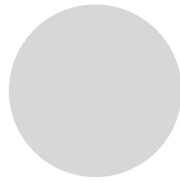
Gambar 1.4.2 Diagram *Mobile-App Use Case*

1.4.3 Website Design

Rancangan website GATEKEEPER-AI dikembangkan sebagai pusat kontrol dan monitoring berbasis web yang digunakan oleh dosen dan administrator untuk

mengelola akses ruangan serta memantau kehadiran mahasiswa secara real-time. Desain website mengedepankan aspek usability, kecepatan akses, serta integrasi dengan *backend system*. Beberapa aspek utama yang menjadi fokus dalam desain yang dibuat :

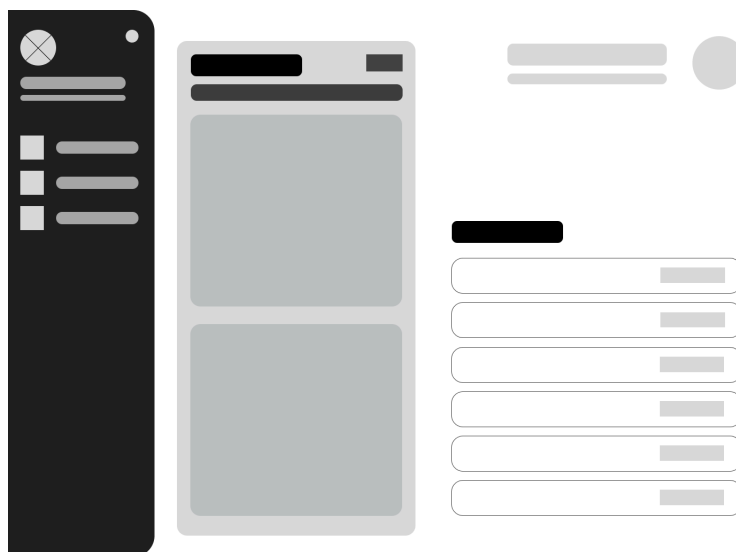
- **Monitoring Real-Time:** Website menampilkan data kehadiran mahasiswa secara langsung dengan refresh rate ≤ 5 detik. Informasi yang ditampilkan mencakup daftar mahasiswa, status kehadiran, timestamp masuk, serta status akses pintu.
- **Manajemen Data Presensi:** Website menyediakan fitur untuk melihat, memfilter, dan mengekspor data presensi berdasarkan kelas, tanggal, maupun mata kuliah. Data diambil langsung dari database terpusat (Granular Database).
- **Kontrol Akses (Remote Override):** Website memungkinkan dosen atau admin untuk membuka atau mengunci pintu secara manual melalui fitur override. Perintah dikirim ke server dan diteruskan ke ESP32 dengan waktu respon ≤ 5 detik.
- **Integrasi dengan Sistem Akademik (SIX):** Website terhubung dengan database jadwal perkuliahan dari SIX untuk memastikan validasi akses dilakukan secara akurat berdasarkan jadwal aktual.
- **Keamanan dan Autentikasi:** Website dilengkapi dengan sistem login berbasis kredensial terverifikasi (SSO ITB) untuk memastikan hanya pengguna berwenang yang dapat mengakses sistem.
- **User Interface yang Intuitif:** Desain UI dibuat sederhana dan mudah dipahami sehingga dosen dapat dengan cepat mengakses informasi penting tanpa pelatihan khusus.



Gambar 1.4.3.1 Splash Screen



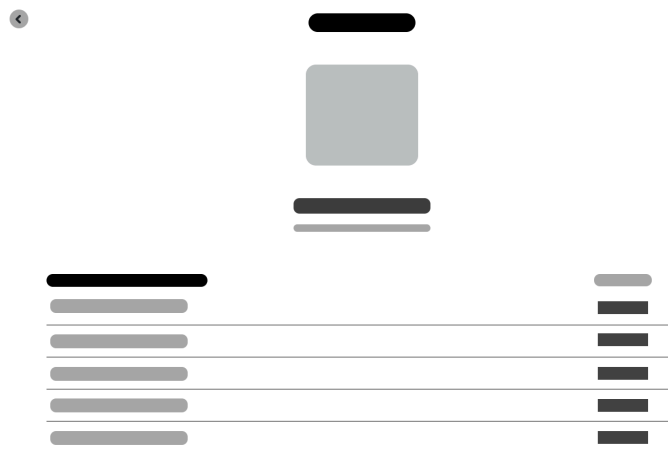
Gambar 1.4.3.2 Login



Gambar 1.4.3.3 Dashboard Mahasiswa



Gambar 1.4.3.4 Daftar Jadwal Mata Kuliah Mahasiswa



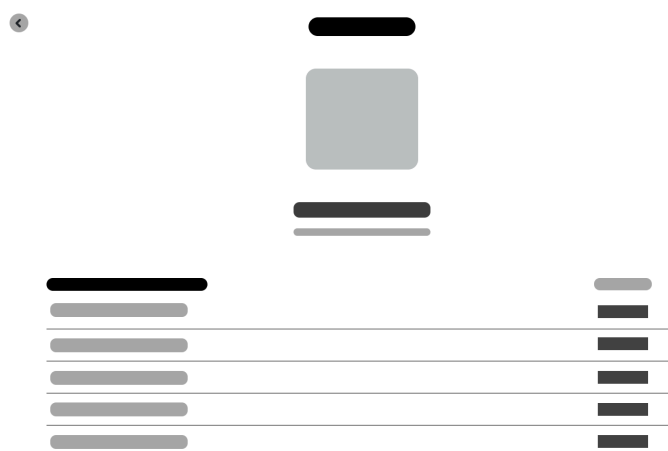
Gambar 1.4.3.5 Rincian Kehadiran Kelas Mahasiswa



Gambar 1.4.3.6 Dashboard Dosen



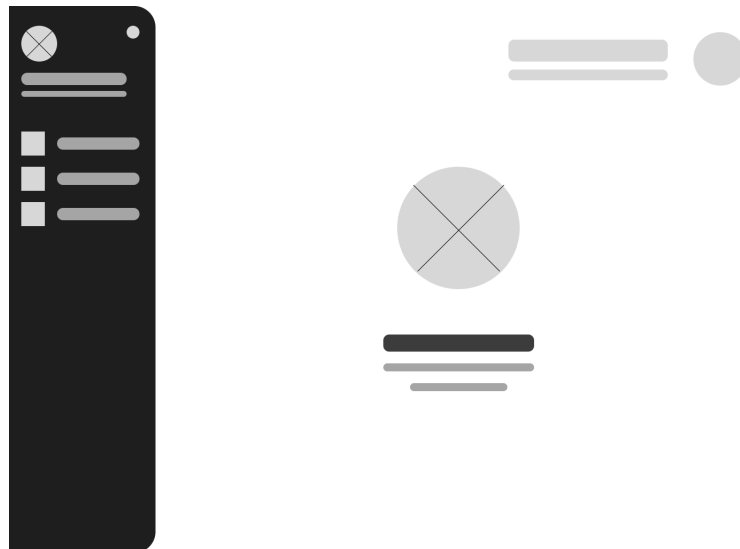
Gambar 1.4.3.7 Jadwal Kelas Dosen



Gambar 1.4.3.8 Rincian Kehadiran Kelas Dosen



Gambar 1.4.3.9 Daftar Mahasiswa



Gambar 1.4.3.10 Profil Pengguna



Gambar 1.4.3.11 Notifikasi

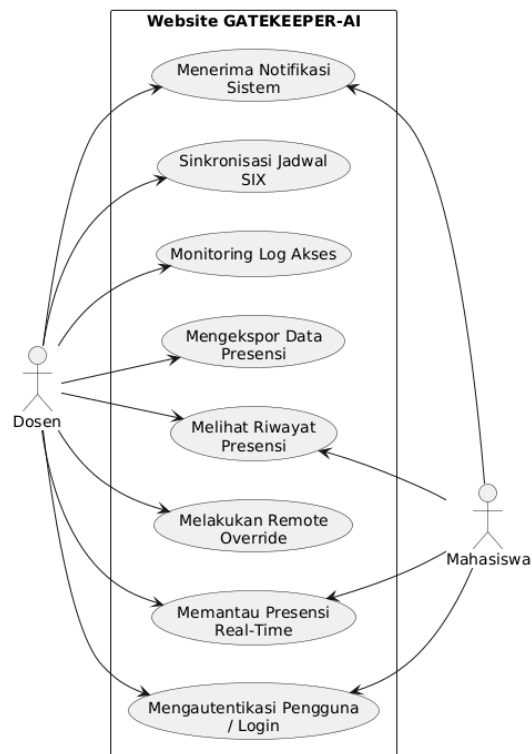
1.4.4 Website Use Case

Bagian ini mendefinisikan interaksi fungsional antara pengguna (Dosen dan Mahasiswa) sebagai aktor dengan *website* dalam ekosistem GATEKEEPER-AI. Kedua aktor ini memiliki akses ke sebagian besar fitur yang sama, dengan pembedaan hak akses khusus pada kontrol fisik pintu ruangan. Dengan beberapa *use case* seperti berikut :

Tabel 1.4.4 Spesifikasi Website Use Case pada GATEKEEPER-AI

ID	Use Case Name	Req	Description
UC-Web-01	Mengautentikasi Pengguna (Login)	R-1, R-9	Dosen dan mahasiswa melakukan login menggunakan kredensial terverifikasi (SSO ITB) untuk mengakses sistem. Sistem memastikan hanya pengguna berwenang yang dapat mengakses fitur website.
UC-Web-02	Memantau Presensi Real-Time	R-4, R-5	Website menampilkan data kehadiran mahasiswa secara real-time, termasuk status hadir/alpha, timestamp, dan aktivitas keluar-masuk.

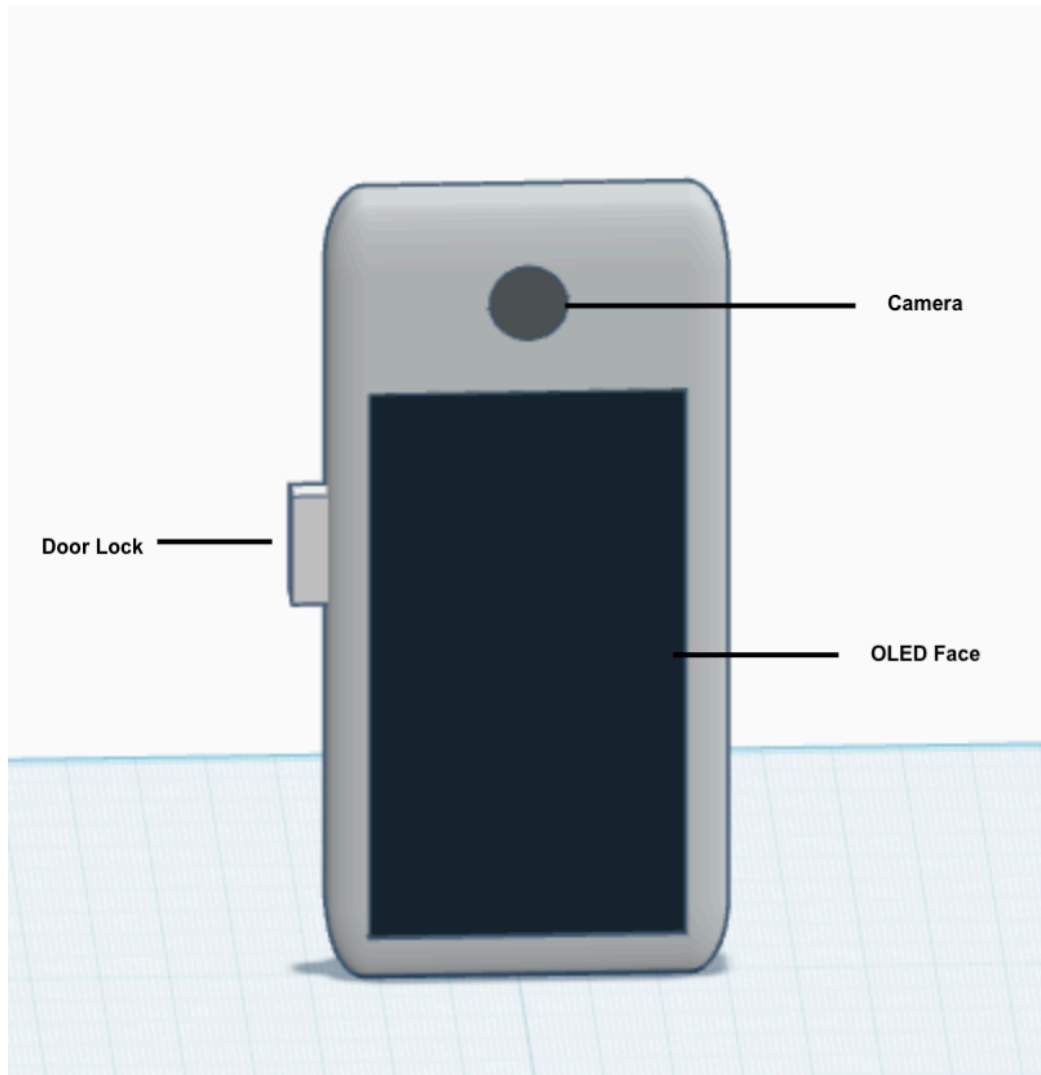
UC-Web-03	Melihat dan Memfilter Data Presensi	R-4	Pengguna dapat melihat dan memfilter data presensi berdasarkan kelas, tanggal, mata kuliah, atau mahasiswa tertentu.
UC-Web-04	Mengekspor Data Presensi	R-6	Website menyediakan fitur untuk mengekspor data presensi dalam format tertentu (misalnya CSV/Excel) untuk keperluan administrasi dan evaluasi.
UC-Web-05	Melakukan Remote Override	R-6	Dosen dapat membuka atau mengunci pintu secara manual melalui website tanpa melalui proses biometrik.
UC-Web-06	Sinkronisasi Jadwal dari SIX	R-3	Website secara otomatis atau manual melakukan sinkronisasi data jadwal perkuliahan dari sistem SIX untuk memastikan validasi akses tetap akurat.
UC-Web-07	Monitoring Log Akses	R-4	Website menampilkan log akses lengkap (NIM, waktu, status akses, ruangan) sebagai bentuk audit sistem.
UC-Web-08	Error Handling & Monitoring Sistem	R-7	Sistem mendeteksi error dan memberikan notifikasi kepada pengguna jika terjadi gangguan sistem.



Gambar 1.4.4 Diagram *Website Use Case*

1.4.5 *IoT Design*

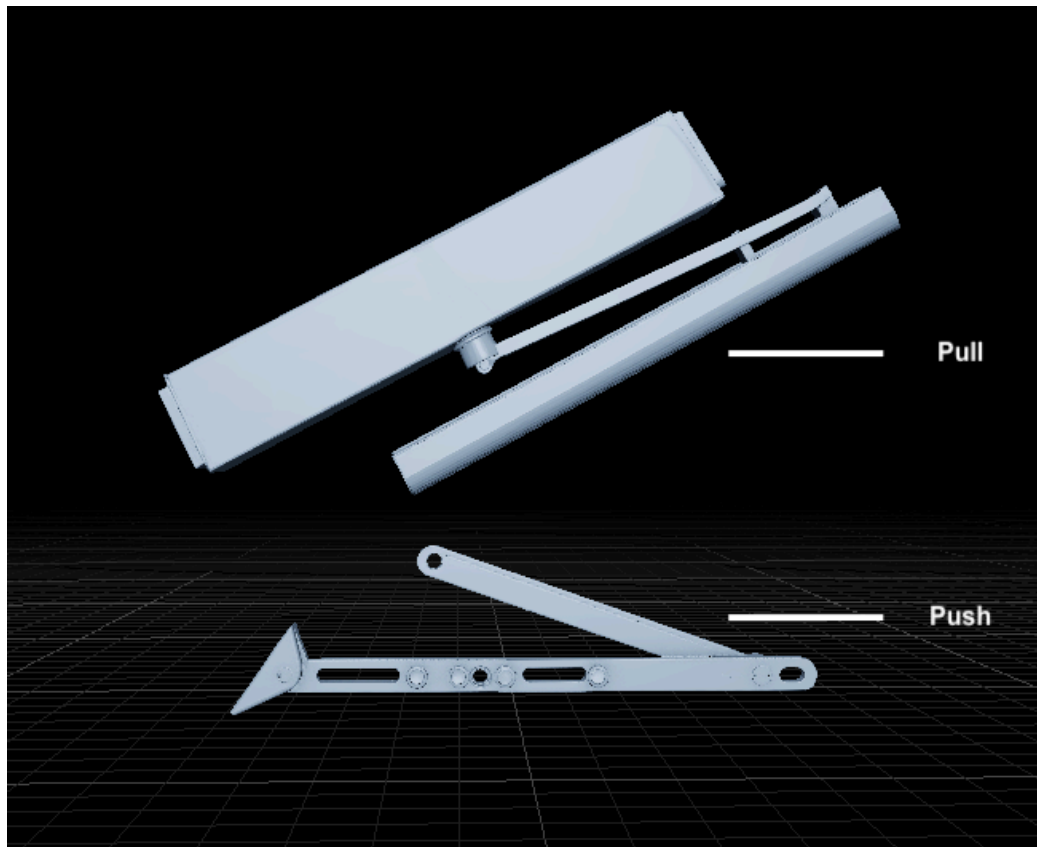
Pada tahap pengembangan ini, kami berfokus pada Design A (Biometric Door Lock) sebagai unit utama antarmuka biometrik, yang kemudian diintegrasikan dengan Design B (Automatic Door Swing) sebagai mekanisme penggerak pintu otomatis.



Gambar 1.4.5.1 IoT Design Door Lock

Design Biometric Door Lock (Fokus Utama)

- Camera (Input Sensor): Terletak di bagian atas untuk menangkap citra wajah sesuai requirement R-11 (jarak 0,2–1 meter). Kamera ini mengirimkan data biometrik secara real-time ke Edge Engine.
- OLED Face (Visual Feedback): Layar ini memberikan umpan balik langsung kepada mahasiswa, seperti status verifikasi "Berhasil" atau instruksi posisi wajah.
- Integrated Lock Indicator: Bagian samping yang menunjukkan integrasi kunci fisik, yang dalam implementasinya akan terhubung secara digital ke unit penggerak pintu.



Gambar 1.4.5.2 IoT Design Door Swing

Design Automatic Door Swing

- Automatic Door Closer Unit: Modul persegi yang menampung motor penggerak dan ESP32. Unit ini mengeksekusi perintah "OPEN" berdasarkan instruksi server (R-12).
- Swing Mechanism (Pull & Push): Lengan mekanis ini dirancang secara modular agar sistem kompatibel dengan berbagai arah bukaan pintu di gedung ITB, baik yang ditarik ke dalam (Pull) maupun didorong ke luar (Push).
- Actuator Control: Mekanisme swing ini bekerja secara sinkron dengan Biometric Door Lock pada Design A, memastikan akses terbuka hanya setelah verifikasi biometrik tervalidasi.

1.4.6 IoT Use Case

Bagian ini mendeskripsikan interaksi antara komponen fisik perangkat IoT (Kamera, ESP32, dan unit gerbang otomatis) dengan pengguna serta sistem

backend. Fokus utamanya adalah pada aliran data biometrik dan eksekusi kendali pintu secara real-time.

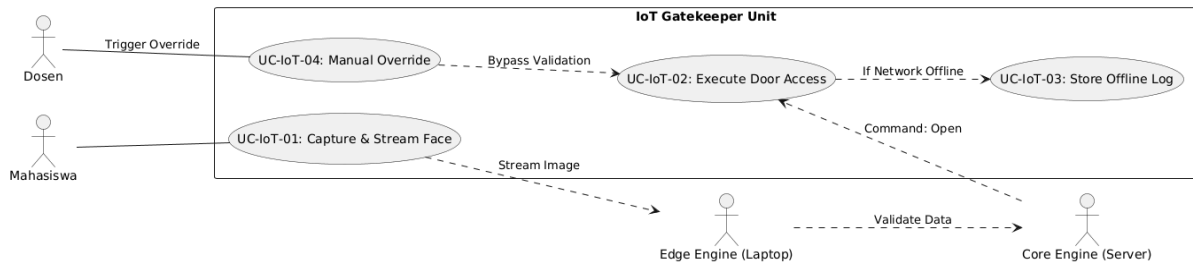
Tabel 1.4.6.1 Aktor dan Sistem yang Terlibat pada IoT GATEKEEPER-AI

Aktor/Sistem	Peran dalam Ekosistem IoT
Mahasiswa	Aktor utama yang memberikan input biometrik (wajah) pada unit IoT untuk verifikasi kehadiran dan akses ruangan.
Dosen/Admin	Aktor yang memiliki otoritas untuk memicu pembukaan pintu secara manual melalui aplikasi tanpa melalui proses verifikasi biometrik.
Edge Engine	Unit pemroses lokal yang melakukan ekstraksi fitur wajah dari stream kamera dan mengirimkan hasil validasi ke Core Engine.
Core Engine (Server)	Sistem pusat yang memvalidasi hak akses berdasarkan jadwal kuliah dan menginstruksikan IoT untuk membuka gerbang.
IoT Unit (ESP32)	Kontroler utama yang mengelola sensor kamera, aktuator pintu otomatis, dan penyimpanan log lokal (offline caching).

Tabel 1.4.6.2 Spesifikasi IoT Use Case pada GATEKEEPER-AI

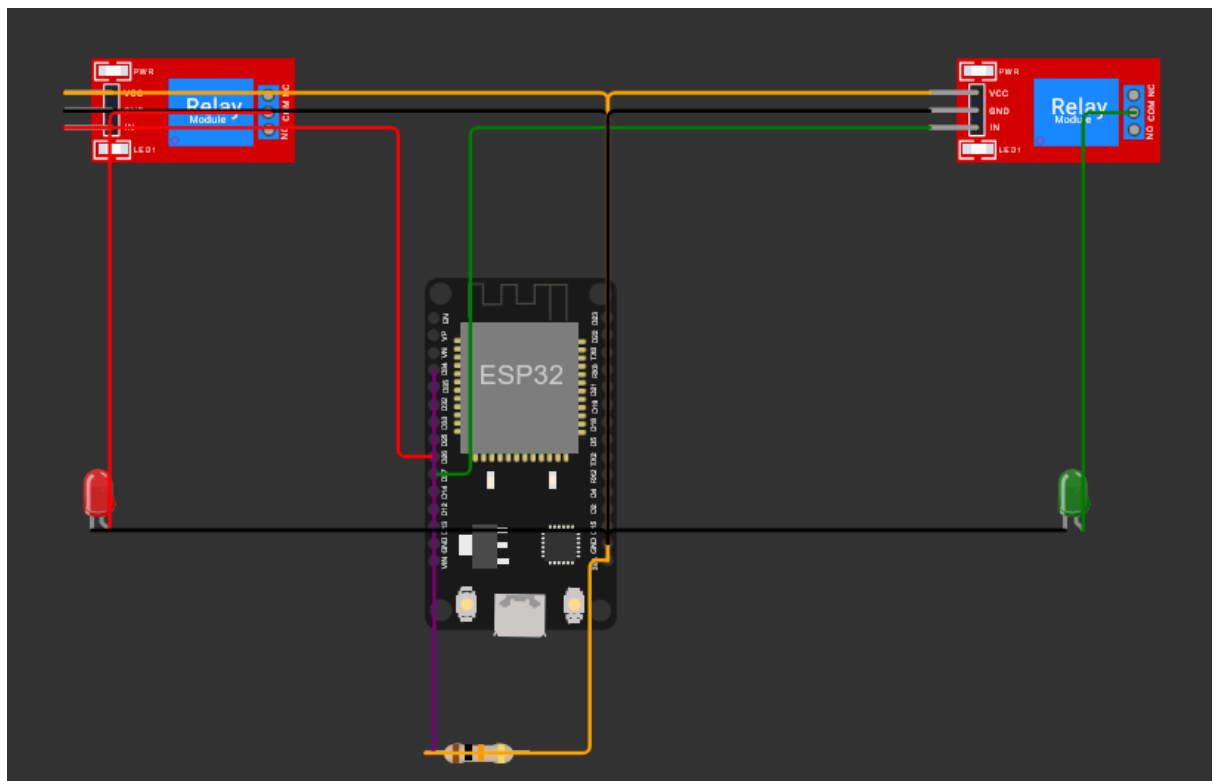
ID	Use Case Name	Req	Description
UC-IoT-01	Capture & Stream Face	R-11	Kamera menangkap citra wajah mahasiswa pada jarak 0,2 – 1 meter dan mengirimkannya ke Edge Engine untuk diproses.
UC-IoT-02	Door Access Execution	R-12, R-14	ESP32 menerima instruksi dari server dan mengaktifkan mekanisme pembuka pintu otomatis (swing gate/solenoid) jika akses diberikan.
UC-IoT-03	Offline Data Caching	R-13	ESP32 secara otomatis menyimpan log presensi ke memori lokal jika koneksi Wi-Fi

ID	Use Case Name	Req	Description
			terputus, memastikan tidak ada data yang hilang.
UC-IoT-04	Manual Override	R-14	Menerima dan mengeksekusi perintah pembukaan pintu darurat yang dikirimkan oleh Dosen melalui aplikasi mobile.



Gambar 1.4.6. Use Case Diagram IoT Gatekeeper-AI

1.4.7 IoT Circuit Design



Gambar 1.4.7. Rancangan IoT Circuit Design

Pada desain ini, *solenoid door lock* dan *swing opener* masing-masing disimulasikan menggunakan lampu LED merah dan lampu LED hijau karena komponennya tidak

tersedia di platform Wokwi dan untuk status pintu disimulasikan oleh magnetic reed sensor.

1.5 Development Testing

1.5.1 Test Planning

Rencana pengujian ini disusun untuk memverifikasi fungsionalitas sistem secara modular (per komponen) dan terintegrasi sebelum sistem diimplementasikan sepenuhnya di lingkungan kampus. Fokus utama pengujian adalah pada akurasi identifikasi wajah, keamanan terhadap manipulasi (anti-spoofing), kecepatan respon perangkat IoT, dan integritas data pada kondisi jaringan tidak stabil.

1.5.1.1 Tujuan Pengujian (*Test Objectives*)

- Memastikan kamera mampu menangkap citra wajah dengan jelas pada jarak operasional 0,2–1 meter (R-11).
- Memvalidasi akurasi model face recognition agar mencapai target $\geq 95\%$.
- Menguji responsivitas ESP32 dalam mengaktifkan mekanisme pembuka pintu otomatis dalam waktu < 2 detik (R-12, R-8).
- Menjamin fungsionalitas offline caching agar log kehadiran tetap tersimpan saat Wi-Fi terputus (R-13).

1.5.1.2 Overview Test Plans

- Lingkungan Pengujian (Testing Environment): Infrastruktur teknis yang mencakup spesifikasi perangkat keras seperti MacBook Monterey sebagai Edge Engine, mikrokontroler ESP32, dan kondisi fisik pencahayaan koridor kampus akan dijelaskan secara mendalam pada sub-bab [1.5.2 Testing Environment](#).
- Identifikasi dan Kriteria Uji (Test Identification & Criteria): Matriks yang merinci setiap item pengujian (seperti akurasi wajah dan latensi pintu) beserta parameter keberhasilan kuantitatifnya (misal: akurasi $> 95\%$) akan dijabarkan pada sub-bab [1.5.3 Test Identification & Criteria](#).

- Metodologi Pengujian (Testing Methodology): Pendekatan sistematis yang digunakan oleh tim, mulai dari Unit Testing pada level kode, Black-box Testing pada fungsionalitas pintu, hingga pengujian keamanan (anti-spoofing) akan dibahas pada sub-bab [1.5.4 Testing Methodology](#).

Tabel 1.5.1 Tabel Testing Plan pada GATEKEEPER-AI

Test ID	Test Item	Test Objective	Test Input	Expected Output	Test Method	Pass Criteria
T01	Face Recognition	Validasi akurasi identifikasi wajah.	Wajah mahasiswa terdaftar.	NIM dikenali dan data dikirim ke server.	Observasi & logging.	Akurasi > 95%.
T02	Liveness Detection	Mencegah kecurangan pakai foto/video.	Foto wajah di layar smartphone.	Akses ditolak, status "Fake Detected".	Spoofing attack simulation.	100% menolak media 2D.
T03	Door Actuator	Menguji respon fisik pintu (R-12).	Perintah "OPEN" dari server.	Mekanisme pintu otomatis terbuka.	Real-time measurement.	Latensi respon < 2 detik.
T04	Offline Log	Validasi resiliensi data (R-13).	Akses saat Wi-Fi dimatikan.	Data tersimpan di memori lokal ESP32.	Fault injection (Wifi Off).	Data Loss = 0% saat sinkronisasi.
T05	Remote Override	Menguji kendali manual dosen.	Tombol "Open" di Mobile App.	Pintu terbuka secara instan.	End-to-end integration test.	Eksekusi berhasil dalam < 5 detik.

1.5.2 Testing Environment

Bagian ini merinci infrastruktur teknis dan kondisi lingkungan yang digunakan selama fase pengembangan dan pengujian sistem GATEKEEPER-AI untuk memastikan hasil yang konsisten dan dapat direplikasi.

1.5.2.1 Perangkat Keras (*Hardware*)

Pengujian menggunakan perangkat keras dengan spesifikasi berikut untuk menjamin performa edge computing dan kontrol aktuator yang stabil:

- Edge Processing Node: MacBook Pro/Air dengan macOS 12 Monterey, digunakan sebagai unit pemroses algoritma Face Recognition lokal.

- Microcontroller Unit: ESP32 DevKit V1 yang bertanggung jawab atas komunikasi REST API dan kendali aktuator.
- Imaging Sensor: Kamera OV2640 (2 Megapixel) untuk penangkapan citra biometrik mahasiswa.
- Door Actuator: Unit Automatic Door Closer/Swing Gate (atau Solenoid Door Lock 12V) sebagai mekanisme kontrol akses fisik.
- Sensor Status: Magnetic Reed Sensor untuk memantau status terbuka/tertutupnya pintu secara real-time.

1.5.2.2 Perangkat Lunak (*Software*)

Lingkungan perangkat lunak disusun untuk mendukung integrasi end-to-end antara IoT dan Cloud:

- Development IDE: Arduino IDE (untuk firmware ESP32) dan Visual Studio Code (untuk backend Python/[Node.js](#)).
- Database & Backend: Firebase Realtime Database sebagai media penyimpanan log dan sinkronisasi status pintu.
- Interface: Aplikasi mobile berbasis Cloud untuk akses remote override oleh dosen.

1.5.2.3 Kondisi Lingkungan Fisik

Pengujian dilakukan pada simulasi lingkungan koridor dan ruang kelas ITB dengan parameter berikut:

- Pencahayaan: Variasi kondisi pencahayaan mulai dari cahaya lampu ruangan normal (± 250 lux) hingga kondisi minim cahaya (< 10 lux) untuk menguji reliabilitas sensor kamera.
- Jarak Operasional: Objek (wajah) diposisikan pada jarak 0,2 meter hingga 1 meter dari lensa kamera sesuai dengan skenario penggunaan di depan pintu kelas.
- Konektivitas: Menggunakan jaringan Wi-Fi lokal untuk mensimulasikan transmisi data JSON antara Edge, ESP32, dan Cloud.

1.5.3 Test Identification & Criteria

Bagian ini menetapkan identitas unik bagi setiap item pengujian beserta kriteria kelulusan (Pass Criteria) yang ketat. Penentuan kriteria ini didasarkan pada Measures of Performance (MoP) yang telah ditetapkan pada tahap awal untuk menjamin kualitas operasional GATEKEEPER-AI.

Tabel 1.5.3 Tabel Test Identification & Criteria pada GATEKEEPER-AI

Test ID	Test Item	Input / Trigger	Parameter yang Diukur	Pass Criteria
T-SYS-01	<i>Face Feature Extraction</i>	Citra wajah dari kamera.	Confidence Score kecocokan wajah.	Skor kecocokan > 90% pada database lokal.
T-SYS-02	<i>Liveness Verification</i>	Upaya penetrasi menggunakan foto/video (2D).	Tingkat penolakan media non-manusia.	100% menolak akses untuk media statis (foto/layar HP).
T-IoT-01	<i>Actuator Response</i>	Instruksi "OPEN" dari server via REST API.	Latensi fisik pergerakan pintu otomatis.	Pintu bergerak terbuka dalam waktu < 2 detik setelah sinyal diterima.
T-IoT-02	<i>Offline Data Integrity</i>	Pemutusan koneksi Wi-Fi selama sesi presensi.	Jumlah log yang tersimpan di memori lokal ESP32.	100% data tersinkronisasi otomatis saat koneksi pulih tanpa korupsi data.
T-LOG-01	<i>Time-Threshold Accuracy</i>	Event "Check-out" mahasiswa melampaui 30 menit.	Presisi waktu perubahan status di database.	Status berubah dari "Hadir" menjadi "Alpha" tepat pada menit ke-31.
T-APP-01	<i>Manual Override Latency</i>	Penekanan tombol "Force Open" pada mobile app.	Waktu transmisi perintah end-to-end.	Perintah dieksekusi oleh hardware dalam waktu < 5 detik.

1.5.4 Testing Methodology

Sistem GATEKEEPER-AI diuji menggunakan pendekatan berlapis untuk memastikan setiap subsistem berfungsi secara individu maupun kolektif. Metodologi pengujian dibagi menjadi empat kategori utama:

1.5.4.1 Unit Testing (Pengujian Terisolasi)

Pengujian dilakukan pada level kode program paling dasar sebelum diintegrasikan ke perangkat keras.

- Algoritma Biometrik: Menguji skrip pengenalan wajah menggunakan dataset foto statis untuk mendapatkan nilai threshold confidence yang optimal.

- Firmware ESP32: Memverifikasi logika perintah OPEN/LOCK pada mikrokontroler menggunakan Serial Monitor tanpa menghubungkannya ke aktuator fisik terlebih dahulu.

1.5.4.2 Black-box Testing (Pengujian Fungsional)

Metode ini berfokus pada pengujian fungsionalitas antarmuka tanpa memperhatikan alur internal kode.

- Skenario Input-Output: Tim memberikan input berupa wajah asli di depan kamera dan mengamati apakah pintu terbuka sesuai harapan.
- Remote Override Test: Menguji tombol pada aplikasi mobile dosen dan mengukur waktu yang dibutuhkan hingga pintu merespons secara fisik.

1.5.4.3 Stress & Load Testing (Pengujian Beban)

Mensimulasikan kondisi beban puncak (peak load) yang sering terjadi di kampus ITB, terutama saat pergantian jam kuliah.

- Verifikasi Massal: Melakukan simulasi 20 mahasiswa yang masuk secara berturut-turut dalam jendela waktu 1–2 menit untuk melihat stabilitas suhu perangkat edge dan latensi respons pintu.
- Concurrent Requests: Mengirimkan beberapa perintah override secara bersamaan dari aplikasi mobile dan dashboard web untuk memastikan tidak terjadi race condition pada server.

1.5.4.4 Security & Vulnerability Testing (Pengujian Keamanan)

Secara spesifik ditujukan untuk memvalidasi fitur anti-spoofing.

- 2D Spoofing Attack: Mencoba melakukan presensi menggunakan foto cetak kualitas tinggi dan video wajah mahasiswa dari layar smartphone untuk memastikan fitur Liveness Detection bekerja menolak akses.
- Fault Injection: Melakukan pemutusan koneksi internet secara tiba-tiba saat proses pengiriman data presensi untuk memvalidasi mekanisme offline caching dan integritas log data.

1.5.5 Test Scenario

1.5.5.1 Langkah-Langkah Pengujian

Skenario pengujian dirancang untuk memverifikasi respons sistem terhadap berbagai interaksi pengguna dan kondisi lingkungan. Setiap skenario mengikuti langkah-langkah sistematis untuk memastikan hasil pengujian dapat divalidasi secara objektif terhadap spesifikasi fungsional.

Tabel 1.5.5.1 Tabel Skenario & Langkah-langkah Pengujian pada GATEKEEPER-AI

No	Skenario Uji	Langkah-Langkah Pengujian	Hasil yang Diharapkan	Status
1	Presensi Normal (Check-in)	1. Mahasiswa berdiri di depan kamera unit IoT pada jam kuliah yang sesuai. 2. Kamera melakukan streaming wajah ke edge engine.	Wajah dikenali, status berubah menjadi "Hadir", pintu otomatis terbuka dalam waktu < 2 detik.	Planned
2	Akses Mahasiswa Salah Jadwal	1. Mahasiswa yang tidak terdaftar pada sesi tersebut mencoba masuk. 2. Sistem melakukan validasi biometrik.	Identitas dikenali, namun sistem menolak akses karena jadwal tidak cocok. Pintu tetap terkunci.	Planned
3	Deteksi Kecurangan (Spoofing)	1. Seseorang mencoba presensi menggunakan foto atau video mahasiswa di smartphone. 2. Sistem menjalankan algoritma liveness detection.	Sistem mendeteksi media 2D, memberikan peringatan "Fake Detected", dan akses ditolak sepenuhnya.	Planned
4	Logika Ghosting (30 Menit)	1. Mahasiswa memindai wajah untuk keluar kelas (Izin). 2. Mahasiswa tidak kembali ke ruangan setelah 30 menit berlalu.	Sistem secara otomatis mengubah status presensi dari "Hadir" menjadi "Alpha" pada database pusat.	Planned
5	Intervensi Dosen (Override)	1. Dosen menekan tombol "Open Door" pada aplikasi mobile dalam kondisi darurat. 2. Perintah dikirimkan ke server dan diteruskan ke ESP32.	Mekanisme pintu otomatis terbuka secara instan meskipun tanpa verifikasi biometrik.	Planned
6	Uji Resiliensi Offline	1. Memutuskan koneksi Wi-Fi pada unit IoT. 2. Mahasiswa melakukan akses masuk seperti biasa.	Log akses tersimpan di memori lokal ESP32. Saat Wi-Fi kembali aktif, data langsung tersinkronisasi ke server.	Planned

1.5.5.2 Overview Test Scenario

Bagian ini memberikan kerangka kerja mengenai bagaimana hasil pengujian diolah, dievaluasi, dan dimitigasi untuk memastikan keberhasilan proyek GATEKEEPER-AI. Detail mengenai metrik

performa hingga prosedur pemulihan sistem akan dijabarkan dalam poin-poin berikut:

- **Metrik Performa (Performance Metrics):** Evaluasi kuantitatif yang membandingkan target parameter pada Deliverable 1 (seperti akurasi wajah > 95% dan latensi < 2 detik) dengan hasil aktual yang dicapai selama pengembangan akan dirinci pada sub-bab [1.5.6 Performance Metrics](#).
- **Evaluasi Hasil Uji (Test Result Evaluation):** Penjelasan mengenai metodologi penilaian status kelulusan modul (PASSED/FAILED) serta analisis akar masalah terhadap setiap temuan selama pengujian akan dibahas pada sub-bab [1.5.7 Test Result Evaluation](#).
- **Prosedur Kegagalan dan Pemulihan (Failure and Recovery Procedure):** Rencana kontingensi teknis untuk menangani kondisi darurat, seperti kegagalan daya, gangguan jaringan Wi-Fi ITB, hingga mekanisme manual override oleh dosen akan dijabarkan pada sub-bab [1.5.8 Failure and Recovery Procedure](#).
- **Mitigasi Risiko Pengembangan (Development Risk Mitigation):** Identifikasi potensi hambatan teknis selama proses pengerjaan (seperti masalah pencahayaan atau overheating) beserta langkah preventifnya akan dirinci pada sub-bab [1.5.9 Development Risk Mitigation](#).

1.5.6 Performance Metrics

Bagian ini menyajikan perbandingan antara target performa yang telah ditetapkan pada tahap Needs Analysis (Deliverable 1) dengan hasil aktual yang diperoleh selama fase pengujian pengembangan. Matriks ini berfungsi sebagai indikator efektivitas sistem dalam memenuhi standar Measures of Performance (MoP) dan Measures of Effectiveness (MoE).

Tabel 1.5.5 Tabel Skenario & Langkah-langkah Pengujian pada GATEKEEPER-AI

Kode	Parameter Performa	Target (Deliverable 1)	Hasil Aktual (Pengembangan)	Keterangan
MoP1	Akurasi Identifikasi Wajah	> 98%	<i>TBD</i>	Fokus pada dataset pencahayaan rendah.
MoP2	Latensi Respon Pintu	< 2 detik	<i>TBD</i>	Pengukuran dari wajah terdeteksi hingga solenoid bergerak.

MoP5	Latensi Sinkronisasi Cloud	< 2 detik	<i>TBD</i>	Kecepatan pembaruan data lokal ke Firebase.
MoE1	Reduksi Proxy Attendance	> 95% penurunan	<i>TBD</i>	Diukur melalui jumlah anomali pada log akses.
MoE3	Akurasi Status Kehadiran	99% akurasi	<i>TBD</i>	Validasi logika time-threshold 30 menit.

Meskipun hasil aktual masih dalam tahap pengumpulan (*TBD*), identifikasi awal menunjukkan bahwa variabel Pencahayaan (*MoP1*) dan Stabilitas Wi-Fi (*MoP3*) akan menjadi faktor penentu utama dalam mencapai target performa. Ketidaksesuaian yang mungkin muncul akan dianalisis lebih lanjut pada bagian evaluasi untuk menentukan apakah diperlukan optimasi algoritma atau peningkatan spesifikasi perangkat keras.

1.5.7 Test Result Evaluation

Evaluasi hasil pengujian dilakukan secara berkala untuk memastikan setiap penyimpangan (*gap*) antara hasil aktual dengan target performa dapat segera diidentifikasi dan diperbaiki. Proses evaluasi didasarkan pada tiga kriteria penilaian utama:

1.5.7.1 Klasifikasi Status Kelulusan

Setiap skenario uji akan dikategorikan ke dalam tiga status berdasarkan tingkat keberhasilannya:

- **PASSED:** Hasil aktual memenuhi atau melampaui seluruh kriteria Pass Criteria yang ditetapkan pada Tabel 1.5.3.1.
- **PASSED WITH NOTES:** Fungsi utama bekerja, namun terdapat defisiensi kecil pada parameter non-kritis (misalnya latensi sedikit melampaui target namun sistem tetap stabil).
- **FAILED:** Fungsi gagal beroperasi atau terdapat celah keamanan kritis (seperti kegagalan Liveness Detection) yang memerlukan iterasi desain segera.

1.5.7.2 Analisis Akar Masalah (Root Cause Analysis)

Jika ditemukan kegagalan (FAILED), tim akan melakukan klasifikasi penyebab berdasarkan temuan di lapangan:

- Defisiensi Desain: Kesalahan pada logika algoritma atau arsitektur sistem.
- Keterbatasan Perangkat Keras: Kemampuan sensor (Kamera) atau mikrokontroler (ESP32) yang mencapai batas operasional.
- Faktor Lingkungan: Gangguan eksternal seperti intensitas cahaya yang terlalu ekstrem atau interferensi sinyal Wi-Fi.

1.5.7.3 Penentuan Iterasi

Hasil evaluasi akan menjadi landasan bagi Project Manager untuk menentukan langkah selanjutnya:

- Optimasi: Melakukan perbaikan kode atau kalibrasi ulang sensor.
- Revisi Spesifikasi: Menyesuaikan target performa jika kendala fisik tidak memungkinkan untuk diatasi tanpa perubahan biaya yang radikal.
- Finalisasi: Jika seluruh skenario kritis berstatus PASSED, sistem dinyatakan layak untuk lanjut ke tahap Engineering Development.

1.5.8 Failure and Recovery Procedure

Prosedur ini disusun untuk menjamin keselamatan pengguna dan keamanan ruangan apabila terjadi kegagalan sistem pada unit GATEKEEPER-AI. Rencana pemulihan ini memastikan bahwa kegagalan teknis tidak menghambat akses darurat maupun operasional dasar kelas.

1.5.8.1 Skenario Kegagalan dan Tindakan Pemulihan

Tabel 1.5.8.1 Skenario Kegagalan dan Tindakan Pemulihan pada GATEKEEPER-AI

Jenis Kegagalan	Dampak Operasional	Prosedur Pemulihan (Recovery)
Kegagalan Daya (Power Outage)	ESP32 mati dan kunci solenoid kehilangan daya.	Sistem dikonfigurasi pada mode Fail-Secure (terkunci) untuk keamanan aset, namun dapat dibuka secara mekanis menggunakan kunci

Jenis Kegagalan	Dampak Operasional	Prosedur Pemulihan (Recovery)
		fisik cadangan yang dipegang oleh dosen/petugas.
Gangguan Jaringan (Wi-Fi Down)	Unit IoT tidak dapat melakukan validasi ke Core Engine secara real-time.	Sistem secara otomatis beralih ke Mode Offline. ESP32 melakukan validasi menggunakan local cache mahasiswa terdaftar dan menyimpan log presensi sementara di memori internal.
Kesalahan Identifikasi (False Rejection)	Mahasiswa sah gagal dikenali oleh sistem biometrik.	Mahasiswa dapat melakukan presensi ulang atau dosen menggunakan fitur Remote Override pada aplikasi mobile untuk membuka pintu secara paksa.
Sistem Hang / Freeze	Perangkat Edge atau ESP32 tidak merespons input.	Implementasi fitur Watchdog Timer pada ESP32 untuk melakukan Auto-Restart sistem jika terdeteksi tidak responsif dalam waktu tertentu.

1.5.8.2 Mekanisme Sinkronisasi Pasca Pemulihan

Setelah konektivitas atau daya pulih, sistem akan menjalankan prosedur sinkronisasi otomatis:

- Integrity Check: Memeriksa kesesuaian antara jumlah log lokal dengan data terakhir di server.
- Batch Upload: Mengirimkan seluruh log yang terkumpul selama masa offline ke database Firebase secara bertahap untuk menghindari beban puncak server.
- Status Update: Memperbarui status kehadiran mahasiswa secara dinamis pada dashboard dosen berdasarkan timestamp asli kejadian akses.

1.5.9 Development Risk Mitigation

Tahap mitigasi risiko pengembangan bertujuan untuk mengidentifikasi potensi hambatan teknis sejak dini dan menyiapkan langkah preventif agar jadwal serta kualitas sistem GATEKEEPER-AI tetap terjaga. Fokus mitigasi ini adalah pada keseimbangan antara performa sistem dan keterbatasan sumber daya perangkat keras yang digunakan.

Tabel 1.5.9 Tabel Manajemen Risiko Development pada GATEKEEPER-AI

Sumber Risiko	Deskripsi Dampak	Strategi Mitigasi	Biaya/Upaya Mitigasi
Akurasi Rendah pada Cahaya Minim	Kamera OV2640 gagal mengekstraksi wajah di koridor yang redup.	Menambahkan modul LED infra merah atau mengaktifkan fitur Auto Gain Control pada firmware kamera.	Upaya: Kalibrasi ulang sensor; Biaya: ±Rp50.000.
Latensi Jaringan Tinggi	Verifikasi tertunda karena koneksi Wi-Fi kampus yang tidak stabil.	Mengoptimalkan ukuran data JSON dan menerapkan mekanisme Adaptive Buffering serta Local Caching pada ESP32.	Upaya: Optimasi skrip API.
Overheating pada Perangkat Edge	Laptop (MacBook) panas saat menjalankan pemrosesan wajah kontinu.	Menggunakan Edge Engine yang hanya aktif saat sensor mendeteksi gerakan, serta optimasi beban komputasi Python.	Upaya: Peninjauan ulang algoritma deteksi.
Overfitting Model AI	Model gagal mengenali variasi wajah (masker/kacamata) mahasiswa ITB.	Memperkaya dataset pelatihan lokal dengan augmentasi data yang mencakup berbagai aksesoris dan sudut wajah.	Upaya: Pengumpulan 100+ sampel data baru.
Kegagalan Sinkronisasi SIX	Jadwal kuliah tidak terupdate tepat waktu di server pusat.	Menyediakan Endpoint sinkronisasi manual dan log	Upaya: Pembuatan dashboard

Sumber Risiko	Deskripsi Dampak	Strategi Mitigasi	Biaya/Upaya Mitigasi
		pemantauan status koneksi ke database SIX.	monitor status.

2. PENUTUP

Fase *Advanced Development* pada sistem GATEKEEPER-AI telah berhasil menerjemahkan rancangan konseptual menjadi spesifikasi teknis, desain fungsional, dan model prototipe yang komprehensif. Melalui analisis persyaratan (*requirement analysis*) yang terperinci, kebutuhan operasional telah diturunkan menjadi metrik yang terukur untuk setiap subsistem, meliputi *Edge Engine*, *Core Server*, perangkat IoT, hingga antarmuka aplikasi (*Mobile* dan *Web*).

Pada tahap desain fungsional dan pengembangan prototipe, interaksi antar-elemen sistem telah dipetakan dengan jelas untuk menangani skenario operasional normal, intervensi manual (*remote override*), hingga penanganan kondisi darurat seperti hilangnya koneksi jaringan (*offline caching*). Perancangan prototipe antarmuka juga telah mengedepankan aspek *usability* untuk memastikan pengguna, baik dosen maupun mahasiswa, dapat berinteraksi dengan sistem secara intuitif.

Selain itu, perencanaan pengujian (*test planning*) yang mencakup *Unit Testing*, *Black-box Testing*, hingga *Security Testing* (khususnya *liveness detection*) telah disusun secara terstruktur. Rencana pengujian beserta strategi mitigasi risiko ini memastikan bahwa setiap komponen sistem siap untuk divalidasi kinerjanya sebelum diimplementasikan secara nyata. Dengan selesainya dokumen Deliverable 2 ini, arsitektur GATEKEEPER-AI telah mencapai tingkat kematangan yang memadai untuk dilanjutkan ke fase *Engineering Design* dan implementasi skala penuh dengan tingkat risiko yang terkendali.

LAMPIRAN

No	Prompt (copy-paste asli)	Tujuan	Bagian yang Dibantu	Penjelasan Hasil
1	Apa perbedaan utama antara <i>functional design</i> dan <i>physical architecture</i> dalam rekayasa sistem?	Mendapatkan pemahaman tentang batasan apa saja yang harus ditulis saat merancang desain fungsional.	1.3 Functional Design	Pemahaman bahwa <i>functional design</i> berfokus pada "apa" yang dilakukan sistem (aliran data, interaksi logika) tanpa terikat pada bentuk perangkat keras spesifiknya, yang mana baru akan dibahas pada arsitektur fisik.
2	Bagaimana aturan peletakan aktor dan <i>system boundary</i> yang benar saat membuat <i>Use Case Diagram</i> (UML)?	Mengetahui standar baku penggambaran <i>use case</i> agar interaksi mahasiswa dan dosen dengan sistem terpetakan dengan benar.	1.4 Prototype Development (Use Case)	Didapatkan penjelasan mengenai kaidah <i>association line</i> (tanpa panah) antara aktor dan <i>use case</i> , serta pentingnya meletakkan batasan sistem (<i>boundary</i>) berbentuk kotak di tengah aktor.
3	Apa saja metode umum yang digunakan untuk <i>liveness detection</i> pada <i>face recognition</i> di <i>edge device</i> ?	Memahami teori dasar <i>anti-spoofing</i> untuk memperkuat dasar perancangan keamanan subsistem kamera.	1.2.2 Komponen yang Sangat Kompleks & 1.5.1 Test Planning	Alternatif metode <i>liveness detection</i> yang dijelaskan meliputi analisis tekstur wajah (2D), deteksi kedalaman (3D), dan deteksi pergerakan (mata berkedip), yang kemudian digunakan sebagai landasan skenario pengujian.
4	Jelaskan perbedaan	Mencari tahu	1.5.4 Testing	<i>Unit testing</i>

	antara <i>black-box testing</i> dan <i>unit testing</i> , serta kapan keduanya digunakan.	metode pengujian perangkat lunak/keras yang paling relevan untuk diimplementasikan.	Methodology	digunakan untuk menguji isolasi kode skrip (seperti fungsi hitung mundur 30 menit), sedangkan <i>black-box testing</i> menguji fungsionalitas akhir (seperti apakah pintu terbuka saat wajah dikenali) tanpa melihat kode internal.
5	Bagaimana konsep dasar <i>offline caching</i> pada mikrokontroler IoT seperti ESP32 ketika kehilangan koneksi Wi-Fi?	Mempelajari mekanisme penyimpanan data sementara dan resiliensi sistem pasca pemulihan jaringan.	1.4.6 IoT Use Case & 1.5.8 Failure and Recovery	Didapatkan konsep penyimpanan data JSON ke memori lokal ESP32 (EEPROM/Flash) yang kemudian dikirimkan secara serentak (<i>batch upload</i>) ketika status koneksi kembali berstatus <i>online</i> .
6	Apa itu <i>Root Cause Analysis</i> (RCA) dalam tahap evaluasi pengujian sistem perangkat keras?	Memahami cara mengevaluasi dan mengklasifikasikan hasil pengujian sistem jika berstatus <i>FAILED</i> .	1.5.7 Test Result Evaluation	RCA adalah metode penyelesaian masalah untuk mengidentifikasi penyebab dasar kegagalan, yang dikategorikan menjadi cacat desain, limitasi <i>hardware</i> , atau gangguan dari faktor lingkungan eksternal.

REFERENSI

Kossiakoff, A., Biemer, S. M., Seymour, S. J., & Pappadakis, D. A. (2011). *Systems Engineering Principles and Practice* (2nd ed.). Hoboken, NJ: John Wiley & Sons.

Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson Education.

International Organization for Standardization. (2015). *Systems and Software Engineering — Life Cycle Processes* (ISO/IEC/IEEE 15288:2015). Geneva, Switzerland: ISO.

A. K. Jain, A. Ross, and S. Prabhakar, “An Introduction to Biometric Recognition,” *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 14, no. 1, pp. 4–20, 2004.

INCOSE, *Systems Engineering Handbook: A Guide for System Life Cycle Processes and Activities*, 4th ed. Wiley, 2015.

M. Rouse, “Internet of Things (IoT),” *TechTarget*, 2020.